Advanced Data Analysis

Mike Nguyen

2021-10-09

Contents

Chapter 1

Prerequisites

```
install.packages("bookdown")
# or the development version
# devtools::install_github("rstudio/bookdown")
```

This book's skeleton is based on several courses. Hence, I'd like to credit professors accordingly.

Course	Professor
Data Analysis 3	Christopher Wikle
Analyzing Unstructured Data	Kunpeng Zhang
Deep Learning	Yann LeCun

Compare to the last book, this book should differ as follows:

Part I

MACHINE LEARNING

Chapter 2

Introduction

Resources

- INTRODUCTION TO MACHINE LEARNING
- An introduction to Statistical Learning
- The Elements of Statistical Learning

Machine learning is also known as statistical learning

- two types of learning:
  - supervised learning: predicting or estimating output based on inputs
  - unsupervised learning: understanding the relationships among inputs without output

3 types of data

- Structured Data (tables form)
- Semi-structured Data (e.g., XML, JSON, HTML)
- Unstructured Data
  - Texts
  - Images: Videos, photos,
  - Networks: Social Networks
- ML is concerned with the the development, the analysis, and the application of algorithm that allow computers to learn
- Learning:
  - A computer learns if it improves its performance at some tasks with experience (i.e., data)
  - Extracting a model of system from the sole observation (or the simulation)

  - A model = some relationships between the variables used to describe the system.

- Goals:
  - Prediction
  - Inference

- Learning if useful when:
  - Human expertise does not exist (navigating on Mars)
  - Humans are unable to explain their expertise (speech recognition)
  - Solution changes in time (routing on a computer network)
  - Solution needs to be adapted to particular cases (user biometrics)

- Applications in different domains:
  - Retail: Market basket analysis, customer relationship management (CRM)
  - Finance: Credit scoring, fraud detection.
  - Manufacturing: Optimization, troubleshooting
  - Medicine: medical diagnosis
  - Telecommunications: Quality of service optimization, routing
  - Bioinformatics: Motifs, alignment WEb mining, search engines.

- **Steps**
  - Data generation: data types, sample size
  - Preprocessing: normalization, missing values, feature selection/extraction
  - Machine Learning: Hypothesis, choice of learning paradigm/algorithm.
  - Hypothesis validation: cross-validation, model deployment.

- Types of machine learning:
  - Supervised: manually label variables
  - Unsupervised:

- Training and testing
  - Training is the process of making the machine learn
  - No free lunch:
    * Training set and testing set come from the same distribution
    * Need to make some assumptions.

- Factors affecting the performance:
  - Types of training provided
  - The form and extend of prior knowledge
  - Type of feedback provided
  - The learning algorithms used

- Two factors:
  - Modeling
  - Optimization
- Algorithms:
  - ML depends on algorithms.
  - The algorithms control the search to find and build the knowledge structures
  - The learning algorithms should extract useful information from training examples.
- Types of learning (algorithms):
  - Supervised learning ($\{X_n \in R^d, y_n \in R\}_{n=1}^N$)
    - Prediction
    - Classification (discrete labels), regression (real values)
  - Unsupervised learning ($\{X_n \in R^d\}_{n=1}^N$)
    - Clustering
    - Probability distribution estimation
    - Finding association (in features)
    - Dimension reduction
  - Semi-supervised learning
  - Reinforcement learning
    - Decision making (robot, chess machine)

2.1 General Framework

- Y = output
- $X = (X_1, ..., X_p)$ = p inputs

Under supervised learning, additive error model:

$$Y = f(x) + \epsilon$$

where

- $f(.)$ = unknown fixed function
- ϵ = random error ($E(\epsilon) = 0; \epsilon \perp X$)

For prediction (blackbox)

$$\hat{Y} = \hat{f}(X)$$

where

- Reducible Error: $\hat{f}(.)$ can be improved (in principle)
- Irreducible Error: ϵ because it's not a function of the inputs, which might contain unknown variables or uncontrolled variation

$$\begin{aligned} E(Y - \hat{Y})^2 &= E[f(X) + \epsilon - \hat{f}(X)]^2 \\ &= [f(X) - \hat{f}(X)]^2 + var(\epsilon) \\ &= \text{reducible} + \text{irreducible} \end{aligned}$$

For inference, $f(X) = E(Y|X)$, but the distribution of Y|X is unknown; hence, must be estimated.

We try to find $\hat{f}$ such that $Y \approx \hat{f}(X) \forall (X, Y)$

Ways to estimate f:

- Parametric
  1. Make an assumption about the function form of $f(X)$
  2. Try to fit data to that specified form (i.e., estimate the parameters) using OLS in regression or MLE in general.
- Nonparametric

Tradeoff between prediction accuracy and model interpretability

Restrictive Model	Flexible Model
More inference More interpretable	More prediction

2.2 Assessing Model Accuracy - Model fit

2.2.1 Continuous output: Mean squared error (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{f}(x_i))^2$$

Generalization Error is a measure of how accurately an algorithm or model is able to predict outcome values for test set.

Tradeoff between flexibility of the method and training MSE , but there is an U-shaped relationship between flexibility and test MSE. This is a manifestation of bias-variance tradeoff

$$E(MSE) = E(y_0 - \hat{f}(x_0))^2 = var(\hat{f}(x_0)) + [bias(\hat{f}(x_0))]^2 + var(\epsilon)$$

where 0 subscript means training data set

- $E(MSE) \geq var(\epsilon)$ because ϵ is irreducible model error and serves as the lower bound on the quality of potential model fit
- More flexible means more variance, but less bias.
- Less flexible (parametric) means less variance, but more bias.

2.2.2 Categorical output: training error rate

Training error rate is the proportion of mistakes in classification from applying $\hat{f}$ to the training set:

$$\frac{1}{n}\sum_{i=1}^{n} I(y_i \neq \hat{y}_i)$$

and **test error rate** is

$$\frac{1}{n}\sum_{i=1}^{n} I(y_0 \neq \hat{y}_0)$$

Bayes classifier is a simple classifier where it assigns each observation to the most likely class, given its predictor values (i.e., conditional probability that $Y = j|x_0$)

In other words, assigns x_i to class j where $P(Y = j|X = x_0)$ is largest

Bayes classifier should produce the lowest possible test error rate, known as Bayes error rate:

$$1 - E_x(max_j[P(Y = j|X)])$$

- where it is greater than 0 if the classes overlap
- analogous to the irreducible error

In reality, we don't know $P(Y|X)$, hence, we try to approximate this conditional probability.

For example, KNN

$$P(Y = j|X = x_0) \approx \frac{1}{K}\sum_{i \in N_0} I(y_i = j)$$

2.2.3 Curse of dimensionality

Example: with a p-dimensional hypercube of unit volume, to capture a fraction (r) of the observations in a subcude, the length of a side of the sub-cube averages to $e_p(r) = r^{1/p}$

$$e_1(.01) = .01; e_1(.1) = .1 e_{10}(.01) = .63; e_{10}(.1) = .8$$

To cover 1% of the observation in 10 dimensional cub, we have to cover 60% of the entire sample space.

Hence, there is no "local" neighborhood in high-dimensions.

2.2.4 Reminder

Linear Regression

$$Y = f(X) + \epsilon$$

where $f(X) = \beta_0 + \sum_{j=1}^{p} X_j \beta_j$ and X_J can be:

- quantitative inputs
- transformations of quantitative inputs
- polynomial terms
- interactions
- Discretize (i.e., dummy) variable of quantitative inputs.

We assume additive independent errors.

Relationship between n and p	Result
$n >> p$	low bias, low predictive variance, fit well with test data
$n \approx p$	substantial variability, overfit, fit poorly with test data
$n < p$	no unique solution, variance is infinite

And we fit to (i.e., find β) to minimize the residual sum of squares (RSS):

$$RSS(\beta) = \sum_{i=1}^{n} (y_i - f(x_i))^2 = \sum_{i=1}^{n} (y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j)^2 = (\mathbf{y} - \mathbf{X})^T (\mathbf{y} - \mathbf{X}) \equiv ||\mathbf{y} - \mathbf{X}||^2$$

which is the square of the L^2 norm

differentiating with respect to β

$$\mathbf{X}^T(\mathbf{y} - \mathbf{X}\ ) = 0$$

then

$$\hat{\beta} = \mathbf{X}^T\mathbf{X}^{-1}\mathbf{X}^T\mathbf{y}$$

and

$$var(\hat{\beta}) = (\mathbf{X}^T\mathbf{X})^{-1}\sigma^2 = (\mathbf{X}^T\mathbf{X})^{-1}var(\epsilon)$$

Prediction

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\beta} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

where

$$\mathbf{H} \equiv \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$$

known as the hat matrix, which projects orthogonally the $\mathbf{y}$ vector onto the subspace of $\mathbf{X}$

From the Gauss-Markov theorem, OLS estimates are unbiased and have the smallest variance among all linear unbiased estimates.

For an etimator $\tilde{\beta}$

$$MSE(\tilde{\beta}) = E(\tilde{\beta} - \beta)^2 = var(\tilde{\beta} + [E(\tilde{\beta}) - \beta]^2 = variance + bias^2$$

Since OLS is an unbiased estimator $\tilde{\beta} = \hat{\beta}$

We could get smaller MSE if we allow the estimator to be biased. Hence, OLS might not be the best variance-bias trade-off.

Inference

$$\hat{\sigma}^2 = \frac{1}{n-p-1}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

assuming that $\epsilon \sim N(0, \sigma^2)$ then

$$\hat{\beta} \sim N(\beta, (\mathbf{X}^T\mathbf{X})^{-1}\sigma^2)$$

$$(n-p-1)\hat{\sigma}^2 \sim \sigma^2\chi^2_{n-p-1}$$

If p is large (i.e., a lot of covariates), you will likely find at least one variable to be significant by chance, thus, you could find a significant partial t-statistics. The F-test does not suffer from this issue because it accounts for the number of predictors.

Issues with linear regression

- Nonlinearity of the response -predictor relationship
- correlated errors
- Non-constant error variance
- high-leverage points
- Collinearity

Rule of thumb:

- parametric methods will tend to outperform non-parametric methods when the number of observations per predictor is small.

2.2.5 Model Assessment

The "generalization" performance of a learning method is how well it predicts independent test data, which is an objective way to choose between models (in terms of **model selection** and **model assessment**)

Suppose we have

- Y = output variable
- X = vector of inputs
- $\hat{f}(X)$ = prediction model estimated form a training set T

Consider errors between Y and $\hat{f}(X)$ through some loss function. For example, **squared-error loss**

$$L(Y, \hat{f}(X)) = (Y - \hat{f}(X))^2$$

or **absolute error loss**

$$L(Y, \hat{f}(X)) = |Y - \hat{f}(X)|$$

2.2.5.1 Types of Error

Test error (or generalization error) is the prediction error over an independent test sample, and is defined as

$$Err_T = E_{X^\circ, Y^\circ}[L(Y^\circ, \hat{f}(X^\circ))|T]$$

where X and Y are assumed to be drawn randomly from their joint population distribution and training set T is fixed

Thus, the test error refers to the error over an independent sample when $f(.)$ is estimated with this specific training set

Expected prediction error (expected test error)

$$Err = E_T E_{X^\circ, Y^\circ}[L(Y^\circ, \hat{f}(X^\circ))|T] = E_T[Err_T]$$

which averages over possible training sets

Training error is the average loss over the specific training sample, given by

$$\bar{err} = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{f}(x_i))$$

Even though the interest is in the test error, Err_T , which is almost impossible without a separate test data set. because within a single data set, we attempt to approximate such a validation step by adjusting the training error (e.g., AIC model selection) or by efficient re-use (cross-validation/bootstrap) of the training sample, but then we can only get good estimates of Err (expected prediction error), not Err_T

The bias-variance tradeoff when considering loss function is related to the performance of the training error as a measure of test error

2.2.5.2 Model Error/ Model Selection

Training error is more optimistic (less than) the true test error

To read more on quantifying this optimism (Hastie et al., 2001) Ch. 7.4 where

the amount that $\bar{err}$ underestimates the true test error depends on how strongly y_i affects its own prediction.

In other words, the larger $cov(\hat{y}_i, y_i)$, the more optimistic the training error is.

The greater the effective number of parameters (less degrees of freedom) (more flexible model), the larger this co variance

(Hastie et al., 2001) Ch. 7.5 showed that most of model selection criterion (e.g., AIC, BIC C_p) try to estimate prediction error by estimating the optimism

and adjusting the training error accordingly to obtain an unbiased estimate of prediction error.

Alternatively, cross-validation and bootstrap methods provide direct estimates of the expected prediction (test) error, Err

Note:

- A model with more effective number of parameters is more flexible

Let

$$\mathbf{y} = (y_1, ..., y_n)'$$

then a linear fitting method can be written as $\hat{\mathbf{y}} = \mathbf{S}\mathbf{y}$

where $\mathbf{S} = n \times n$ matrix depending on the input vectors x_i, but not on y_i

For example, in linear regression $\mathbf{S} = \mathbf{X}(\mathbf{X}'\mathbf{X})^{-\mathbf{1}}\mathbf{X}'$

Then the effective number of parameters (df) is

$$df(\hat{\mathbf{y}}) = df(\mathbf{S}) = trace(\mathbf{S})$$

which is the sum of the diagonal elements fo $\mathbf{S}$

If $\hat{\mathbf{y}}$ arises from an additive-error model, $Y = f(X) + \epsilon$, with $var(\epsilon) = \sigma_\epsilon^2$, then

$$df(\hat{\mathbf{y}}) = \frac{\sum_{i=1}^{n} cov(\hat{y}_i, y_i)}{\sigma_\epsilon^2}$$

2.2.5.3 Cross-validation

If you have a large data set, you could randomly divide it into a training set (to fit the model) and a validation (or holdout, or test) set (to evaluate the test error rate relative to your loss function of choice such as MSE). But it has 2 issues:

- It can be highly variable
- With fewer observations to fit the model, the statistical models perform worse, and validation set error rate may overestimate the test error rate

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.0.5
```

```
## Warning: package 'ggplot2' was built under R version 4.0.5
```

```
## Warning: package 'tibble' was built under R version 4.0.5
```

```
## Warning: package 'readr' was built under R version 4.0.5
```

```
## Warning: package 'dplyr' was built under R version 4.0.5
```

```
library(ISLR2)
train <- sample(392, 120) # select random subset of 120 out of 392
Auto <- Auto %>% drop_na()
lm.fit <- lm(mpg ~ horsepower , data = Auto , subset = train)

attach(Auto)
mean((mpg - predict(lm.fit, Auto))[-train] ^ 2) # MSE in the validation set (not the training se
```

```
## [1] 26.5254
```

```
# Quadratic regression
lm.fit2 <- lm(mpg ~ poly(horsepower, 2), data = Auto, subset = train)
mean((mpg - predict(lm.fit2, Auto))[-train] ^ 2) # MSE
```

```
## [1] 20.88781
```

```
# Cubic regression
lm.fit3 <- lm(mpg ~ poly(horsepower, 3), data = Auto, subset = train)
mean((mpg - predict(lm.fit3, Auto))[-train] ^ 2) # MSE
```

```
## [1] 20.92144
```

2.2.5.3.1 Leave-One-Out CV (LOOCV) A special case of K-fold CV when K = n

$$CV_{(n)} = \frac{1}{n}\sum_{i=1}^{n} MSE_i$$

where $MSE_i = (y_i - \hat{f}^{-i}(x_i))^2$ (alternative loss function could also be used)

- LOOCV has low bias as an estimate of the expected test error rate, but very high variance
- Individual MSE's in LOOCV are highly correlated because they pretty much have the same data set expect one observation.
- LOOCV is expensive to implement

```
Auto <- Auto %>% drop_na()
library(tidyverse)
glm.fit <- glm (mpg ~ horsepower , data = Auto) # allows to be passed to cv.glm()
coef(glm.fit)
```

```
## (Intercept)  horsepower
##  39.9358610  -0.1578447
```

```
# exactly the same as glm without binomial link
lm.fit <- lm(mpg ~ horsepower , data = Auto)
coef(lm.fit)
```

```
## (Intercept)  horsepower
##  39.9358610  -0.1578447
```

```
library(boot)
```

```
## Warning: package 'boot' was built under R version 4.0.5
```

```
glm.fit <- glm (mpg ~ horsepower , data = Auto)
cv.err <- cv.glm (Auto , glm.fit)
cv.err$delta # cross-validation estimate for the test error
```

```
## [1] 24.23151 24.23114
```

```
# the first delta is the standard CV estimate, the second delta is a bias-corrected ve


# automate to different polynomial
cv.error <- rep (0, 10)
for (i in 1:10) {
    glm.fit <- glm(mpg ~ poly(horsepower , i), data = Auto)
    cv.error[i] <- cv.glm(Auto , glm.fit)$delta[1] # cross-validation estimate for the
}
cv.error # a drop from lienar to quadratic fit
```

```
##  [1] 24.23151 19.24821 19.33498 19.42443 19.03321 18.97864 18.83305 18.96115
##  [9] 19.06863 19.49093
```

2.2.5.3.2 K-Fold Cross-Validation We randomly split the data set into K equal size parts (or folds). Each fold is held out and the model is trained on the remaining K -1 folds and then the loss function of choice is evaluated on the fold that was held out

Consider $k = 1, \ldots, K$ folds

let $\hat{f}^{-k}(x)$ denote the fitted function computed with the k-th fold removed. We compute the loss based on the samples in the k-th fold.

For example, with squared-error loss, we compute MSE_k for the observations in the k-th fold, $k = 1, \ldots, K$. The K-fold cross-validation estimate is

$$CV_{(K)} = \frac{1}{K} \sum_{k=1}^{K} MSE_k$$

Empirically, good choices for K are 5 or 10.

```
cv.error.10 <- rep(0, 10)
for (i in 1:10) {
    glm.fit <- glm(mpg ~ poly(horsepower, i), data = Auto)
    cv.error.10[i] <- cv.glm(Auto, glm.fit, K = 10)$delta[1] # K-fold cross validation
}
cv.error.10
```

```
##  [1] 24.15545 19.47923 19.61763 19.45716 19.00392 19.30588 19.09349 19.54975
##  [9] 18.75923 18.96160
```

2.2.5.3.3 Generalized Cross-Validation Recall PRESS statistics

$$\frac{1}{n}\sum_{i=1}^{n}[y_i - \hat{f}^{-i}(x_i)]^2 = \frac{1}{n}\sum_{i=1}^{n}[\frac{y_i - \hat{f}(x_i)}{1 - S_{ii}}]^2$$

where S_{ii} is the i-th diagonal element of $\mathbf{S}$ (or the i-th element of the "hat matrix" diagonal in linear regression). Hence, we only have to fit the model once.

Generalized cross-validation (GCV)is a similar approximation when the trace of $\mathbf{S}$ can be computed more easily than the individual elements S_{ii}

$$GCV(\hat{f}) = \frac{1}{n}\sum_{i=1}^{n}[\frac{y_i - \hat{f}(x_i)}{1 - trace(\mathbf{S})/n}]^2$$

2.2.5.3.4 CV and Classification Common loss function is 0-1 loss where the error rate is defined as

$$L(G, \hat{G}(X)) = I(G \neq \hat{G}(X))$$

where $\hat{G}(X)$ is the highest estimated conditional probability given the inputs.

Alternatively, we could also use deviances that factor in the estimated conditional probability directly. If $p_k(X) = P(G = k|X)$ and $\hat{G}(X) =$ the k that gives the maximum $\hat{p}_k(X)$

$$L(G, \hat{p}(X)) = -2\sum_{k=1}^{K} I(G = k)\log\hat{p}_k(X) \equiv -2\log\hat{p}_G(X)$$

which is -2 times the log-likelihood (i.e., the deviance).

Note:

- CV must be applied to the entire sequence of modeling steps (i.e., samples must be removed before any selection or filtering steps are applied so long as these steps involve the outputs

  - An exception: if there was an unsupervised step in the process associated with the inputs. Since it's unrelated to the outputs, we don't need to do CV on this component.

2.2.5.4 Bootstrapping

- To quantify the uncertainty associated with a given estimator
- Get new data sets by sampling from the observed data set many times with replacement
- Sample B different sets and calculate the quantity of interest about which you want to do inference $\hat{\alpha}^{*1}, ..., \hat{\alpha}^{*B}$
- Then, you look at the distribution of samples and apply Monte Carlo procedures to do inference
- For example, you want to get the standard error

$$SE_B(\hat{\alpha}) = \sqrt{\frac{1}{B-1}\sum_{r=1}^{B}(\hat{\alpha}^{*r} - \frac{1}{B}\sum_{r'=1}^{B}\hat{\alpha}^{*r'})^2}$$

- In principle, you could evaluate model error, but it often overfits because the training and test samples contain some of the same observations, which can be improved by "leave-one-out" bootstrap procedures, but there are still large training set bias problems with the bootstrap.
- (Hastie et al., 2001) give an estimator that helps reduce this bias.
- In general, cross-validation is more effective for evaluating model performance, and bootstrapping is more effective when evaluating the properties of estimators (for inference).
- Rule of thumb for "big data" prediction problem, we can combine cross-validation and hold-out validation:
  - Use K-fold CV on the training data to choose the parameters that control flexibility of the model
  - Use the entire training data to fit the model selected by CV
  - Use a large hold-out sample to evaluate the test error from this fitted model (compare to the CV error).

```
summary(Portfolio)
```

```
##        X                    Y
##  Min.   :-2.43276   Min.   :-2.72528
##  1st Qu.:-0.88847   1st Qu.:-0.88572
##  Median :-0.26889   Median :-0.22871
##  Mean   :-0.07713   Mean   :-0.09694
```

```
##  3rd Qu.: 0.55809   3rd Qu.: 0.80671
##  Max.   : 2.46034   Max.   : 2.56599
```

```
alpha.fn <- function(data, index) {
    X <- data$X[index]
    Y <- data$Y[index]
    (var(Y) - cov(X, Y)) / (var(X) + var(Y) - 2 * cov(X, Y)) # statistic of interest
}

alpha.fn(Portfolio, 1:100)
```

```
## [1] 0.5758321
```

```
alpha.fn(Portfolio, sample(100, 100, replace = T)) # randomly select 100 obs from 1:100 with repl
```

```
## [1] 0.4746685
```

```
# Another faster way
boot(Portfolio, alpha.fn, R = 100) # R = number of replicates (samples)
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Portfolio, statistic = alpha.fn, R = 100)
##
##
## Bootstrap Statistics :
##      original       bias    std. error
## t1* 0.5758321 -0.003300184  0.09376732
```

Another example: SE estimate for linear model

```
boot.fn <- function(data,index){
    coef(lm(mpg ~ horsepower, data = data, subset = index))
}
boot.fn(Auto, 1:392)
```

```
## (Intercept)  horsepower
##  39.9358610  -0.1578447
```

```
boot.fn(Auto, sample(392,392,replace =T))
```

```
## (Intercept)  horsepower
##  40.2593224  -0.1588387
```

```
boot(Auto, boot.fn,1000)
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
```

```
##
##
## Call:
## boot(data = Auto, statistic = boot.fn, R = 1000)
##
##
## Bootstrap Statistics :
##       original        bias    std. error
## t1* 39.9358610  0.0388923935 0.853686410
## t2* -0.1578447 -0.0005485667 0.007302794
```

```
# there is a difference between the 2 SE estimates
summary(lm(mpg ~ horsepower, data = Auto))$coef
```

```
##               Estimate  Std. Error   t value      Pr(>|t|)
## (Intercept) 39.9358610 0.717498656  55.65984 1.220362e-187
## horsepower  -0.1578447 0.006445501 -24.48914  7.031989e-81
```

```
# Quadratic example
# boot.fn <- function(data,index){
#     coef(
#         lm(mpg ~ horsepower + I(horsepower^2),
#            data = data, subset = index)
#     )
# }
# boot(Auto, boot.fn, 1000)
# summary(
#     lm(mpg ~ horsepower + I(horsepower^2), data = Auto)$coef
# )
```

The difference in SE estimates from bootstrapping and from a linear model comes from:

1. LM estimates σ^2 suing the RSS, which relies heavily on the linear model being correct. In other words, for example, if we have non-linearity in our data, the residuals (e) will be inflated, and leads to $\hat{\sigma}^2$ being inflated as well.
2. LM assumes x_i fixed and variability comes from $var(\epsilon_i)$

2.3 Improving Prediction Accuracy and Interpretability

If we have

1. large number of predictors
2. highly dependent predictors
3. low interpretability

then, we can either

1. Subset Selection
2. Shrinkage Methods (also known as "*regularization*"): force (shrink) p predictors' parameters towards zero to reduce variance. It can also be used for variable selection
3. Dimension Reduction: Project the p predictors in an M-dimensional subpsace where $M < p$ (i.e., compute M different linear combination or projection of the p predictors and use them as new predictors). (e.g., principal component analysis)

2.3.1 Subset Selection

- Also known as Variable Selection (for more detail click the hyperlink)

2.3.1.1 Best Subset Selection

- Fit all p one by one, then all $p(p-1)/2$ two predicotrs models, etc. to have 2^p possible models. Then you find the best one using
  - C_p
  - AIC
  - BIC
  - Adjusted R^2
  - cross-validation error measure
- A possible algorithm is the leaps and bounds algorithm by Furnival and Wilson (1974) (up to 40 p)
- Usually, we do best-subset first then apply cross-validation

```
library(tidyverse)
library(ISLR2)
head(Hitters) # check dataset
```

```
##                   AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
## -Andy Allanson      293   66     1   30  29    14     1    293    66      1
## -Alan Ashby         315   81     7   24  38    39    14   3449   835     69
## -Alvin Davis        479  130    18   66  72    76     3   1624   457     63
## -Andre Dawson       496  141    20   65  78    37    11   5628  1575    225
## -Andres Galarraga   321   87    10   39  42    30     2    396   101     12
## -Alfredo Griffin    594  169     4   74  51    35    11   4408  1133     19
##                   CRuns CRBI CWalks League Division PutOuts Assists Errors
## -Andy Allanson       30   29     14      A        E     446      33     20
## -Alan Ashby         321  414    375      N        W     632      43     10
## -Alvin Davis        224  266    263      A        W     880      82     14
```

```
## -Andre Dawson       828  838    354      N        E     200      11       3
## -Andres Galarraga    48   46     33      N        E     805      40       4
## -Alfredo Griffin    501  336    194      A        W     282     421      25
##                    Salary NewLeague
## -Andy Allanson         NA         A
## -Alan Ashby         475.0         N
## -Alvin Davis        480.0         A
## -Andre Dawson       500.0         N
## -Andres Galarraga    91.5         N
## -Alfredo Griffin    750.0         A
```

```
# check missing values
sum(is.na(Hitters$Salary)) # 59 missing data poitns
```

```
## [1] 59
```

```
# eliminate missing values
Hitters <- Hitters %>%
    drop_na(Salary)

# Best subset seleciton (by RSS)
library(leaps)
regfit.full <- regsubsets(Salary ~ ., Hitters)
# summary(regfit.full)
```

2.3.1.2 Stepwise Selection

- With very large numbers of p, stepwise selection can be less computational intensive
- Forward stepwise selection
  - can be used when $p > n$, but only models up to order $n - 1$
- Backward stepwise selection
  - can't be applied when $p > n$
- Hybrid stepwise
- When applying cross-validation, to account fro the variability, we apply the **one-standard-error rule**
  - Evaluates cross-validation criteria (MSE) for each model size
  - Select the smallest model where the estimated test error is within one SE of the lowest point on the curve (i.e.more parsimonious model)

2.3.2 Shrinkage Methods

- Constrain the Least Squares solution in a way that regularizes (shrinks) the coefficients towards 0.
- This leads to biased estimates, but reduce variance

2.3.2.1 Ridge regression

- can also deal with multicolinnearity
- Adding a constant to the diagonal of the $\mathbf{X}^T\mathbf{X}$ matrix to get well-behaved (low variance) regression estimates even when $\mathbf{X}^T\mathbf{X}$ is nearly singular.

In LS regression, we minimize the residual sum of squares

$$RSS = \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2$$

whereas in ridge regression, we estimate $\hat{\beta}^R$ that minimize

$$\sum_{i=1}^{n}(y_i - \beta - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda\sum_{j=1}^{p}\beta_j^2$$

where λ = tuning parameter

$\lambda\sum_{j=1}^{p}\beta_j^2$ is an l_2-norm shrinkage penalty that is small if the β are near 0.

Minimizing this objective function subject to this constraint forces $\hat{\beta}^R$ to be closer to 0 than the LS estimates.

- When $\lambda = 0$, we get the LS estimates
- When $\lambda \to \infty$, the coefficients go to 0.

Because in the l_2-norm does not contain the intercept (i.e, we don't shrink the intercept), we have to center all the covariates by removing the mean of each column before we do the regression ($\hat{\beta}_0 = \bar{y}$).

$$\hat{\beta}^R = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

To estimate λ, we use cross-validation

- Ridge estimator is not scale-invariant, so coefficients change substantially depending on the scale of the covariates. Hence, before performing ridge regression, we have to scaled and centered covariates (i.e., after removing the mean, divided each column of $\mathbf{X}$ by its standard deviation.
- λ allows for a continuous range of bias-variance trade-offs.

  - As λ increases, the variance decreases but the bias increases. Hence, it works well when LS estimates have high variance.

- Possible even when $p > n$
- Under Bayesian framework, if $y_i \sim N(\beta_0 + x_i^T\beta, \sigma^2)$ is the likelihood and the parameters have the independent prior distributions $\beta_j \sim N(0, \tau^2)$ with known τ^2, σ^2, then $\lambda = \sigma^2/\tau^2$ (the ratio of regression to prior variances), and the ridge estimate is the posterior mode (and mean, since the posterior is Gaussian)
- The ridge estimates are related to principal component regression. When principal components associated with the predictors, ridge regression shrinks the low variance principal component the most.

2.3.2.2 Lasso regression

- Even though ridge regression shrinks the parameters to 0, unless $\lambda = \infty$, it cant set any parameter exactly equal 0. Hence, Tibshirani (1996) proposed lasso to handle this problem

The lasso coefficients, $\hat{\beta}^L$ are those that minimize the objective function

$$\sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda\sum_{j=1}^{p}|\beta_j|$$

The only difference between ridge and lasso is the penalty term:

- Lasso considers l_1 norm penalty (given by $||\beta||_1 = \sum_j |\beta_j|$)
- Ridge considers l_2 norm penalty (given by $||\beta||_2 = \sqrt{\sum_j \beta_j^2}$) similar to Euclidean distance

When tuning parameter (λ) is large enough, some coefficients are 0 (are dropped from the model)

There is no closed form for the lasso estimates $\hat{\beta}^L$, the choice of λ can be derived from cross-validation

Note:

- Lasso outperforms ridge when there are redundant predictors, but ridge outperforms lasso when all included predictors are needed
- Lasso shrinks each parameter by approximately the same amount, while ridge shrinks proportionally
- Lasso equivalent Bayesian interpretation is that: if the prior distribution on the parameters β_j are independent double exponential (Laplace) distribution with mean 0 and scale parameter λ, the posterior mode for β

is the lasso solution (but no the posterior mean); the Laplace prior puts more "mass" at 0, giving more a priori belief that the coefficients are 0.

2.3.2.3 Comparison

$$\min_{\beta} \sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij})^2 \text{ subject to } \begin{cases} \sum_{j=1}^{p} |\beta_j| \leq s \text{ (lasso)} \\ \sum_{j=1}^{p} \beta_j^2 \leq s \text{ (ridge)} \\ \sum_{j=1}^{p} I(\beta_j \neq 0) \leq s \text{ (best subset)} \end{cases}$$

All seek to minimize the RSS subject to constraints on the parameters.

In the best subset method, no more than s coefficients can be nonzero, and can't be computed for large p

2.3.2.4 Cross-validation Fitting

Steps:

1. Choose a "grid" of λ values
2. COmpute the cross-validation error for each vlaue of λ
3. Select the λ for which the cross-validation error is smallest (one-standard error rule)
4. Refit the model using all available observations for the selected tuning parameter.

2.3.2.5 Generalization

We could generalize the objective function to

$$\sum_{i=1}^{n}(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p} |\beta_j|^q$$

for $q \geq 0$, and $q = 0, 1, 2$ correspond to subset selection lasso, and ridge respectively

We could estimate q from the data, but it isn't worth.

Elastic net penalty (= combination of ridge and lasso) will be an alternative

The penalty component in the objective function is the priori distribution on the β under Bayesian framework (e.g., a normal prior = ridge regression and a Laplace prior = lasso). Hence, we can use **stochastic search variable selection (SSVS)** to consider model selection George and McCulloch (1993)

SSVS is a hierarchical mixture of normal distribution

$$\beta_j|\gamma_j \sim \gamma_j N(0, c_j\tau_j^2) + (1 - \gamma_j)N(0, \tau_j^2)$$

$$\gamma_j \sim^{idd} Bern(\pi_j)$$

where $\gamma_j = 1$ means the j-th variable is in the model with probability π_j

We would want τ_j to be small so that when $\gamma_j = 0$, it's reasonable to estimate β_j close to 0.

We would want c_j to be large so that we are more likely to get a non-zero $\beta)j$ when $\gamma_j = 1$ (which is analogous to ridge regression)

An alternative to SSVS produce is **spike and slab** which replaces the second mixture distribution with a delta function at 0 (analogous to lasso)

2.3.3 Dimension Reduction

Let $Z_1, \dots, Z_M$ represent $M < p$ linear combination of the original p predictors:

$$Z_m = \sum_{j=1}^{p} \phi_{jm} X_j, m = 1, \dots, M$$

for constants $\phi_{1m}, \dots, \phi_{pm}$

Consider the least squared linear regression model

$$y_i = \theta_0 + \sum_{m=1}^{M} \theta_m z_{im} + \epsilon_i, i = 1, \dots, n$$

Hence, we will try to find an optimal way to choose ϕ_{jm} so that we will have better results than the least squared solution on the original inputs

Special note

$$\begin{aligned}\sum_{m=1}^{M} \theta_m z_{im} &= \sum_{m=1}^{M} \theta_m \sum_{j=1}^{p} \phi_{jm} x_{ij} \\ &= \sum_{j=1}^{p} \sum_{m=1}^{M} \theta_m \phi_{jm} x_{ij} \\ &= \sum_{j=1}^{p} \beta_j x_{ij}\end{aligned}$$

Hence,

$$\beta_j = \sum_{m=1}^{M} \theta_m \phi_{jm}$$

This means the reduced dimension regression is a special case of OLS except that the parameters β_j are constrained.

Hence, when $M = p$ and Z_m are all linearly independent (no constraints), the reduced dimension regression is equivalent to the OLS . The only benefit is that Z_m's are orthogonal to each other

Similarly to the Shrinkage Methods, the constrained solution is biased, but typically leads to lower variance in the fitted coefficients - especially when $M << p$

2.3.3.1 Principal Component Regression

Steps:

1. Find the linear combination of the inputs that accounted for the largest fraction of variance of the inputs
2. Find another linear combination of the inputs that accounted for the next highest amount of variation (subject to being orthogonal to the first)
3. Continue the process

Mathematically, principal components could also be obtained using **singular value decomposition** (SVD) fo the centered data matrix

Let $\mathbf{X}$ $(n \times p)$ be the centered data matrix, then

$$\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^T$$

where $\mathbf{U}$ and $\mathbf{V}$ are $n \times p$ and $p \times p$ orthogonal matrices

$\mathbf{D}$ (**singular values**) is $p \times p$ diagonal matrix with diagonal entries $d_1 \geq d_2 \geq ... \geq d_p \geq 0$

Then,

$$\mathbf{X}^T\mathbf{X} = \mathbf{V}\mathbf{D}^2\mathbf{V}^T$$

which is the eigen-decomposition of $\mathbf{X}^T\mathbf{X}$ and of the sample covariance matrix $\mathbf{S} = \frac{\mathbf{X}^T\mathbf{X}}{n}$ up to the factor n

And the columns of $\mathbf{V}$ are **eigenvectors** and correspond to the **principal component weights**

Hence, the new variables are

$$\mathbf{z}_1 = \mathbf{X}\mathbf{v}_1, ..., \mathbf{z}_M = \mathbf{X}\mathbf{v}_M$$

where $\mathbf{v}_m$ is the m-th column of $\mathbf{V}$

These $\mathbf{z}$'s are the principal components.

The elements of $\mathbf{v}_m$ are ϕ's in the dimension reduction summary

$$var(\mathbf{z}_m) = \frac{d_m^2}{n}$$

Thus, large (small) singular values correspond to directions in the column space of $\mathbf{X}$ that have large (small) variance

Then principle component regression basically construct the first M principal components (i.e., $Z_1, ..., Z_M$) and uses these components as predictors int the linear regression model fit by least squares

Assumption:

- Directions in which $X_1, ..., X_p$ have the most variation are the directions that are most associated with Y (not always the case, but typically effective). In other words, these X's are mostly associated with Y (best predict), but we cannot be certain that X's variation causes Y's variation.

Notes:

- While capturing most of the variance from inputs, since $M << p$ we don't run into the problem of overfitting
- Another benefit is that the new predictors are orthogonal ($\mathbf{z}_j^T \mathbf{z}_k = 0, j \neq k$), which in the LS estimation we typically run into the problem of collinearity
- Even though PCR uses a low-dimensional set of predictors, no model selection was performed because we only translate all p variables into a linear combination that makes the new PC inputs. Hence, it's closely related to Ridge regression than Lasso regression or Best Subset Selection. Seeing Singular Value Decomposition
  - $\mathbf{X}\hat{\beta}^R = \sum_{j=1}^{p} \mathbf{u}_j \frac{d_j^2}{d_j^2+\lambda} \mathbf{u}_j^T \mathbf{y}$
  - where $\mathbf{u}_j$ corresponds to the j-th column of $\mathbf{U}$ from the SVD
  - Since LS estimate gives $\mathbf{X}\hat{\beta} = \mathbf{U}\mathbf{U}^T\mathbf{y}$, ridge regression goes to the LS solution when $\lambda = 0$
  - The singular vectors that have smaller variation (i.e., smaller d_j^2) are shrunk more than those with larger variation
- Rule of thumb: standardize the predictors before performing the Singular Value Decomposition to generate the principle components.
  - If we don't standardize, the predictors with the largest scale typically account for the most variance, regardless of their importance relative to the output.

2.3.3.2 Partial Least Squares

- Find linear combinations of the data (or directions in the **X** matrix) that maximize **variance** and are **orthogonal**
- Find the weight just like [Principle Component Regression], but PCR in a unsupervised way because the choice of these directions is not dependent on Y. Hence, might not provide good predictions of Y
- PLS is a supervised way to do dimension reduction that finds new features $Z_1, ..., Z_M$ that are linear combinations of the original inputs, but it selects these in a away to account for the covariability withe the response Y

Steps to calculate the PLS directions ($Z_m = \sum_{j=1}^{p} \phi_{jm} X_j$)

1. Standardize the p predictors
2. For Z_1, set each ϕ_{j1} = the coefficient from the simple linear regression of Y on X_j (this is proportional to the correlation between Y and X_j)
3. Calculate Z_2, by calculating the residuals of a regression of each variable (separately) on Z_1 (call these e_j), which corresponds to the info remaining that has not been accounted for by Z_1
4. Set ϕ_{j2} = the coefficient from the simple linear regression of Y on each e_j to get Z_2
5. Repeat M to get $Z_1, ..., Z_2$

Comparison to PCR: Hastie et al. (2001) (chapter 3.5)

The m-th PC direction coefficients $\mathbf{v}_m$ are the α that solve

$$\max_{\alpha}[var(\mathbf{X}\ )]$$

subject to $||\alpha|| = 1, \alpha^T \mathbf{S} \mathbf{v}_l = 0, l = 1, ..., m-1$ (the last condition ensures that each $\mathbf{z}_m = \mathbf{X}\mathbf{v}_m$ is uncorrelated with all of the previous $\mathbf{z}_l$)

where $\mathbf{S}$ is the sample covariance matrix associated with $\mathbf{X}$

The m-th PLS direction $\hat{\phi}_m$ have the α that solves:

$$\max_{\alpha}[corr^2(\mathbf{y}, \mathbf{X}\ )var(\mathbf{X}\ )]$$

subject to $||\alpha|| = 1, \alpha^T \mathbf{S} \hat{\phi}_l = 0, l = 1, ..., m-1$

Notes:

- The extra component in PLS helps make the weight more relatable to Y (but sometimes it's more efficient, but sometimes it isn't)
- Both X and Y should be standardized before performing PLS

- The choice of M is made based on Cross-validation
- Although PLS tries to maximize both the correlation to the output and the variance in the inputs, the variance tends to dominate and PLS behaves very similarly to ridge regression and PCR
- The use of the output to supervise the selection of the reduced dimension can reduce bias, but increase variance (bias-variance trade-off), and reduce the added value of the approach relative to PCR
- PLS is closely related to canonical correlation analysis (CCA) in multivariate statistic, which seeks to find a reduced set of linear combinations of inputs and linear combination of outputs that maximize the correlation (e.lg., finding the coefficient α, β s.t

$$\max_{\alpha,\beta}[corr(\mathbf{Y}\ , \mathbf{X}\ )]$$

where a set of M with orthogonality constraints

- In high-dimensional space ($p \approx n; p > n$), models are easily overfitted (too flexible). Hence, they might appear better than they really are. Hence, to deal with high dimensional data, we use less flexible models (i.e., those that are regularized or constrained), such as Best Subset Selection, Ridge regression, Lasso regression, Principal Component Regression, Partial Least Squares. But we still need to consider their issues
  - Large p always has collinearity; Best Subset Selection or Lasso regression can't guarantee the chosen model is a scientific one
  - p-values are not considered under high-dimensional regressions
  - Tuning parameter selection is utmost important for good predictive performance
  - The test error tends to increase as the dimensionality of the problem increases (Curse of dimensionality)

Chapter 3

Spline Regression

Beyond Linearity (including transformation of variables), we have arbitrary basis expansion (where the basis functions are dependent on X)

3.1 Basis Function Representations

Let the k-th basis function of X be denoted by $b_k(X), k = 1, \dots, K$ which is function that maps from the p-dimensional real numbers to the real line.

Then, the linear functional representation of $f(X)$ given by these basis functions is

$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k b_k(X)$$

Equivalently

$$\mathbf{Y} = \mathbf{B}\beta +$$

where $\mathbf{B}$ is the $n \times (K+1)$ matrix columns given by a vector of 1 for the intercept and $\mathbf{b}_k = (b_k(x_1), \dots, b_k(x_n))', k = 1, \dots, K$

Then, we have

$$\hat{\beta} = (\mathbf{B}^T\mathbf{B})^{-1}\mathbf{B}^T\mathbf{y}$$

if the basis functions are **orthogonal**, then we have a special case $\hat{\beta} = \mathbf{B}^T\mathbf{y}$

Some common basis functions;

- Original linear model: $b_k(X) = X_k, k = 1, \dots, p$
- Polynomials: $b_k(X) = X_j^2; X_j^3; X_j X_m$
- Non-linear transformations: $b_k(X) = \log(X_j); \sqrt{X_j}$
- Step-functions: $b_k(X) = I(c_k \leq X \leq v_k)$, an indicator for a region of X
    1. Breaking the region up into non-overlapping regions and step-function response
    2. Example: for a single X, $b_0(X) = I(X < c_1), \dots, b_{K-1}(X) = I(c_{K-1} \leq X \leq c_K)$ where $c_1, \dots, c_k$ are cut points (i.e., making dummy variables and piece wise constant regression fits)
- Other: sines and cosines, wavelets, empirical basis functions

Another representation

More info can be found on CMU stat

Definition: a k-th order spline is a piecewise polynomial function of degree k, that is continuous and has continuous derivatives of orders 1,…, k -1, at its knot points

Equivalently, a function f is a k-th order spline with knot points at $t_1 < ... < t_m$ if

- f is a polynomial of degree k on each of the intervals $(-\infty, t_1], [t_1, t_2], ..., [t_m, \infty)$
- $f^{(j)}$, the j-th derivative of f, is continuous at $t_1, ..., t_m$ for each j = 0,1,…,k-1

A common case is when k = 3, called cubic splines. (piecewise cubic functions are continuous, and also continuous at its first and second derivatives)

To parameterize the set of splines, we could use **truncated power basis**, defined as

$$g_1(x) = 1 g_2(x) = x ... g_{k+1}(x) = x^k g_{k+1+j}(x) = (x - t_j)_+^k$$

where j = 1,…,m and $x_+ = \max\{x,0\}$

However, now software typically use B-spline basis.

3.2 Regression Splines

- Spline basis functions are built upon polynomial regression and piecewise constant regression
    - We fit piece wise polynomial regressions, but add some continuity and smoothness constraints.

Example:

$$y_i = \begin{cases} \beta_{01} + \beta_{11} + \beta_{21} x_i^2 + \beta_{31} x_i^3 + \epsilon & \text{if } x_i < c \\ \beta_{02} + \beta_{12} + \beta_{22} x_i^2 + \beta_{32} x_i^3 + \epsilon & \text{if } x_i \geq c \end{cases}$$

where c is a **knot** point

The more flexible we want our model to be, the more knots to be added.

If we have K knots through the range of X, we fit $K+1$ different (cubic or any or constants) polynomials.

To restrict piece coming together at the knot points, we add the constraint that the fitted curve must be **continuous**.

We constrain the first and second derivatives to be continuous for **smoothness**.

If the polynomials are cubic, then it's cubic spline.

Each constraint that we add reduces the number of parameters to be fit (frees up df), and less complex

A cubic spline with K knots has $K+4$ df

We can rewrite the cubic spline into the general basis notation

$$b_1(X) = X; b_2(X) = X^2; b_3(X) = X^2; b_4(X) = h(X, \xi_1); b_5(X) = h(X, \xi_2), \ldots, b_{K+3}(X) = h(X, \xi_K)$$

where

$$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x - \xi)^3 & \text{if } x > \xi \\ 0 & \text{otherwise} \end{cases}$$

and ξ is the knot. Fitting this model with an intercept requires estimating $K+4$ regression coefficients (i.e., $K+4$ df)

Notes:

- Splines have high variance at the outer range of the predictors.
  - Natural splines are regression splines with extra constrains on the boundary that mitigates this variance problem by requiring the function to be linear in the regions where X is smaller than the smallest knot, and larger than the largest knot.
  - To decide number and place of knots (K)
    - To choose number of K, people usually do cross-validation

* Given K knots, we locate them uniformly in the quantile domain of X
  - Ex: $K = 3$, a knot would be placed at the 25th, 50th, and 75th percentiles of the variable X.
  - Alternatively, we can place knots based on local data abundance.

Another representation

To estimate the regression function $r(X) = E(Y|X = x)$, we can fit a k-th order spline with **knots** at some prespecified locations $t_1, ..., t_m$

Regression splines are functions of

$$\sum_{j=1}^{m+k+1} \beta_j g_j$$

where

$\beta_1, ..\beta_{m+k+1}$ are coefficients $g_1, ..., g_{m+k+1}$ are the truncated power basis functions for k-th order splines over the knots $t_1, ..., t_m$

To estimate the coefficients

$$\sum_{i=1}^{n}(y_i - \sum_{j=1}^{m} \beta_j g_j(x_i))^2$$

then regression spline is

$$\hat{r}(x) = \sum_{j=1}^{m+k+1} \hat{\beta}_j g_j(x)$$

If we define the basis matrix $G \in R^{n\times(m+k+1)}$ by

$$G_{ij} = g_j(x_i)$$

where $i = 1, .., n$, $j = 1, .., m + k + 1$

Then,

$$\sum_{i=1}^{n}(y_i - \sum_{j=1}^{m} \beta_j g_j(x_i))^2 = ||y - G\beta||_2^2$$

and the regression spline estimate at x is

$$\hat{r}(x) = g(x)^T \hat{\beta} = g(x)^T (G^T G)^{-1} G^T y$$

3.3 Natural splines

A natural spline of order k, with knots at $t_1 < ... < t_m$, is a piecewise polynomial function f such that

- f is polynomial of degree k on each of $[t_1, t_2], ..., [t_{m-1}, t_m]$
- f is a polynomial of degree $(k-1)/2$ on $(-\infty, t_1]$ and $[t_m, \infty)$
- f is continuous and has continuous derivatives of orders 1,.„, k -1 at its knots $t_1, .., t_m$

Note

natural splines are only defined for odd orders k.

3.4 Smoothing splines

We are interested in a function $g(x)$ and try to minimize $RSS = \sum_{i=1}^{n}(y_i - g(x_i))^2$

But we wouldn't want to overfit (because $g(x)$ that minimizes this RSS will try to interpolate all of the y_i), then we want to a smooth $g(x)$ that minimizes RSS:

$$\sum_{i=1}^{n}(y_i - g(x_i))^2 + \lambda g''(x)^2 dx$$

where

- λ is a non-negative smoothness
- $g(.)$ is a **smoothing spline**

Notes:

- RSS here is a *loss function*
- second term is a *smoothness penalty term*
  - Second derivative controls the smoothness of the function (smaller in absolute value means smoother)
  - Parameter λ controls the tradeoff between the smoothness and fit
- When $\lambda = 0$, the function is just like linear function (not smooth at all)
- When $\lambda \to \infty$, then g approaches a straight line that passes through the training data

- The smoothing spline function corresponds to a cubic polynomial basis with knots at the unique values of $x_1, ..., x_n$ with continuous first and second derivatives at each knot
    1. $g(x)$ that minimizes the smoothing spline constraint is just a natural cubic spline with knots at the unique x observations
    2. λ controls the level of shrinkage for smoothing splines, but there is none in regression spline
    3. At first glance, we might think that smoothing splines might have too many degrees of freedom since we have knots at every data points (i.e., $K = n$). But λ controls roughness and make the **effective degrees of freedom** much smaller (as λ increases from $0 \to \infty$, df_λ decreases from $n \to 2$)

Let $\hat{\mathbf{y}}_\lambda$ be the n-vector containing the fitted values for the smoothing spline function for a given λ, then

$$\hat{\mathbf{y}}_\lambda = \mathbf{S}_\lambda \mathbf{y}$$

which means that it is linear in $\mathbf{y}$

According to Hastie et al. (2001) (section 5.4) showed

$\mathbf{S}_\lambda = \mathbf{N}(\mathbf{N}^T\mathbf{N} + \lambda\ _N)^{-1}\mathbf{N}^T$ where $\{N\}_{ij} = b_j(x_i)$ and

$\{\ \}_{jk} = \int b_j''(t)b_k''(t)dt$

The effective degrees of freedom to be trace of $\mathbf{S}_\lambda$ (**smoother matrix** because it smooths the responses in $\mathbf{y}$) is

$$df_\lambda = \sum_{i=1}^{n} \{\mathbf{S}_\lambda\}_{ii}$$

$\mathbf{S}_\lambda$ is related to other types of smoothing in Local Regression and Kernel Smoothing

To choose λ, we can use cross-validation (could also leverage the relationship between λ and effective degrees of freedom).

If we can write a **linear predictor**, we can use a simplified LOOCV formula:

$$\begin{aligned} RSS_{CV}(\lambda) &= \sum_{i=1}^{n} (y_i - \hat{y}_\lambda^{(-i)}(x_i))^2 \\ &= \sum_{i=1}^{n} [\frac{y_i - \hat{y}_\lambda(x_i)}{1 - \{\mathbf{S}_\lambda\}_{ii}}]^2 \end{aligned}$$

where we only have to fit the model once for each λ

3.5 Multidimensional Splines

We can extend the above one-dimensional spline models to higher dimensions using **tensor products** to form higher dimensional basis functions

Consider $X \in R^2$ and

- basis functions $b_{1j}(X_1), j = 1, \dots, K_1$ for representing coordinate X_1
- $b_{2k}(X), k = 1, \dots, K_2$ for coordinate X_2

then the *tensor product basis* is:

$$g_{jk}(X) = b_{1j}(X_1)b_{2k}(X_2)$$

where $j = 1, \dots, K_1, k = 1, \dots, K_2$

which can be used as

$$f(X) = \sum_{j=1}^{K_1}\sum_{k=1}^{K_2}\beta_{jk}g_{jk}(X)$$

Then, we seek to minimize

$$\sum_{i=1}^{n}(y_i - f(x_i))^2 + \lambda J[f]$$

where J is the penalty function for the dimension of the problem

In our example ($X \in R^2$), then

$$J[f] = \int\int [(\frac{\partial^2 f(x)}{\partial x_1^2})^2 + 2(\frac{\partial^2 f(x)}{\partial x_1 \partial x_2})^2 + (\frac{\partial^2 f(x)}{\partial x_2^2})^2]dx_1 dx_2$$

Similar to the one-dimensional case

- $\lambda \to 0$ gives an interpolating function that goes through all points
- $\lambda \to \infty$ gives a solution approaching the least squares plane
- We seek a λ that compromises between these extremes

This splines method is known as **thin-plates splines**.

The solution to the thin-plate spline minimization gives

$$f(x) = \beta_0 + \beta^T x + \sum_{j=1}^{n}\alpha_j b_j(x)$$

where $b_j(x) = ||x - x_j||^2 \log ||x - x_j||$ (known as **radial basis functions**)

Additional splines basis functions could be

- B-spline

Local Regression and Kernel Smoothing are related to the spline basis regression formulations.

Another representation

These estimators use a regularized regression over the natural spline basis: placing knots at all points $x_1, ...x_n$

For the case of cubic splines, the coefficients are the minimization of

$$||y - G\beta||_2^2 + \lambda \beta^T \Omega \beta$$

where $\Omega \in R^{n \times n}$ is the penalty matrix

$$\Omega_{ij} = \int g_i''(t) g_j''(t) dt,$$

and i,j = 1,..,n

and $\lambda \beta^T \Omega \beta$ is the **regularization term** used to shrink the components of $\hat{\beta}$ towards 0. $\lambda > 0$ is the tuning parameter (or smoothing parameter). Higher value of λ, faster shrinkage (shrinking away basis functions)

Note
smoothing splines have similar fits as kernel regression.

	Smoothing splines	kernel regression
tuning parameter	smoothing parameter λ	bandwidth h

3.6 Application

```
library(tidyverse)
library(caret)
```

```
## Loading required package: lattice
```

```
## Warning: package 'lattice' was built under R version 4.0.5
```

```
##
## Attaching package: 'lattice'
```

```
## The following object is masked from 'package:boot':
##
##     melanoma
```

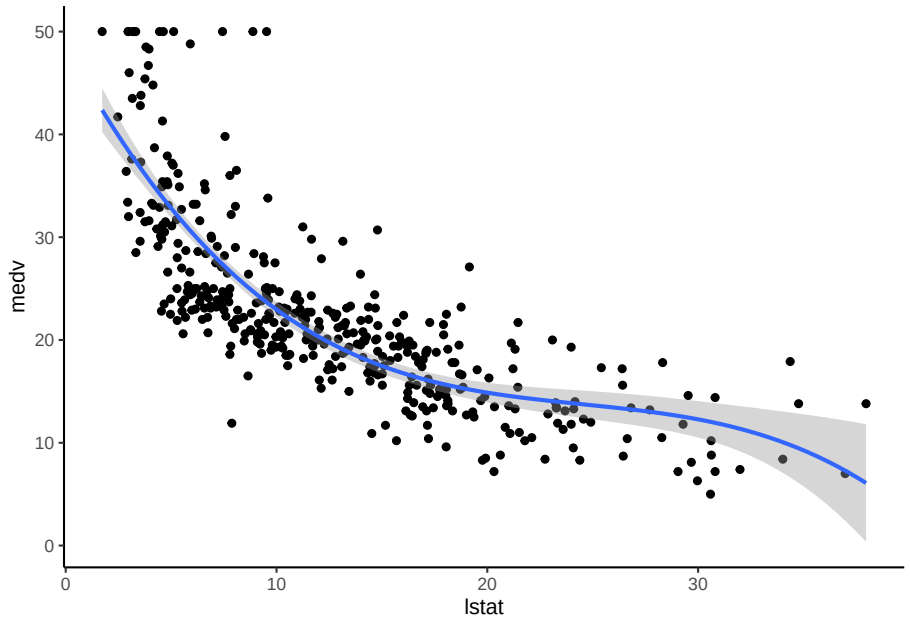
```
##
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
##
##     lift
```

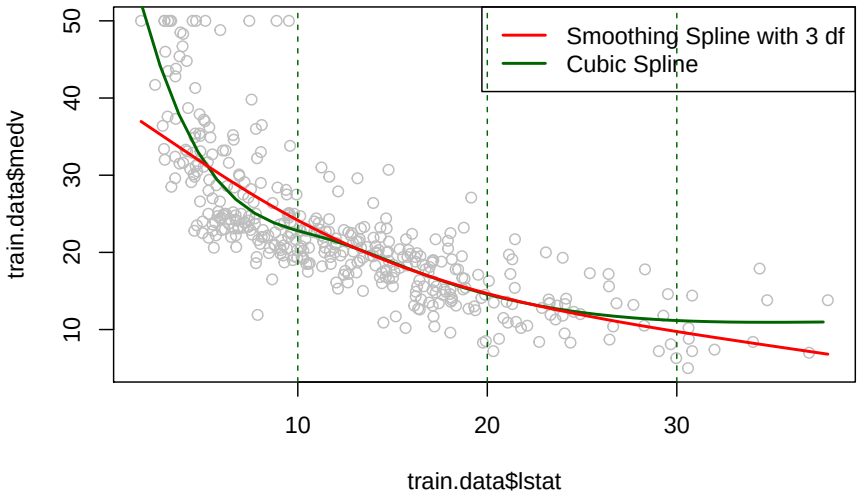
```
theme_set(theme_classic())

# Load the data
data("Boston", package = "MASS")
# Split the data into training and test set
set.seed(123)
training.samples <- Boston$medv %>%
  createDataPartition(p = 0.8, list = FALSE)
train.data  <- Boston[training.samples, ]
test.data <- Boston[-training.samples, ]

knots <- quantile(train.data$lstat, p = c(0.25, 0.5, 0.75)) # we use 3 knots at 25,50,and 75 quar

library(splines)
# Build the model
knots <- quantile(train.data$lstat, p = c(0.25, 0.5, 0.75))
model <- lm (medv ~ bs(lstat, knots = knots), data = train.data)
# Make predictions
predictions <- model %>% predict(test.data)
# Model performance
data.frame(
  RMSE = RMSE(predictions, test.data$medv),
  R2 = R2(predictions, test.data$medv)
)
```

```
##       RMSE        R2
## 1 5.317372 0.6786367
```

```
ggplot(train.data, aes(lstat, medv) ) +
  geom_point() +
  stat_smooth(method = lm, formula = y ~ splines::bs(x, df = 3))
```




```
attach(train.data)
#fitting smoothing splines using smooth.spline(X,Y,df=...)

fit1<-smooth.spline(train.data$lstat,train.data$medv,df=3 ) # 3 degrees of freedom
#Plotting both cubic and Smoothing Splines
plot(train.data$lstat,train.data$medv,col="grey")
lstat.grid = seq(from = range(lstat)[1], to = range(lstat)[2])
points(lstat.grid,predict(model,newdata = list(lstat=lstat.grid)),col="darkgreen",lwd=
#adding cutpoints
abline(v=c(10,20,30),lty=2,col="darkgreen")
lines(fit1,col="red",lwd=2)

legend("topright",c("Smoothing Spline with 3 df","Cubic Spline"),col=c("red","darkgreen
```


Smoothing Spline with 3 df
Cubic Spline
train.data$medv
train.data$lstat
10
20
30
10
20
30
40
50

Chapter 4

Kernel Smoothing

- also known as kernel regression
- The idea is that we fit a separate, but simple, model at each selected point x_0 by using only those observation that are "close" to x_0 such that $\hat{f}(X)$ is smooth in R^p
- The "localization" is accomplished by a **weighting function** (or **kernel**) $K_\lambda(x_i, x_i)$ that assigns a weight to x_i based on its distance from x_0
  - This kernel depends on a parameter λ that defines the "width" or neighborhood" of the kernel
- This method requires less training than traditional regression models (typically, only λ need be chosen), but are **memory-based methods** in that one needs all the training data any time we want to make a prediction at a location x_0

4.1 One-Dimensional Kernel Smoothers

Under k-nearest neighbor, we try to fit

$$\hat{f}(x) = Ave(y_i | x_i \in N_k(x))$$

where $N_k(x)$ refers to the set of k neighbors nearest the point x in squared distance.

Alternatively, we can consider **"Nadaraya-Watson" kernel-weighted average** for our estimate

$$\hat{f}(x_0) = \frac{\sum_{k=1}^{n} K_\lambda(x_0, x_i) y_i}{\sum_{i=1}^{n} K_\lambda(x_i, x_i)}$$

where the kernel is given by

$$K_\lambda(x_0, x) = D(\frac{|x - x_0|}{\lambda})$$

The difference is that now we have a **continuous** function, as compared to **discrete** under the nearest neighbor fit (which can come in handy in some cases).

Examples of the kernel function

1. Epanechinikov quadratic kernel (has no continuous derivatives at the boundary of its support)

$$D(t) = \begin{cases} \frac{3}{4}(1 - t^2) & \text{if } |t| \leq \lambda \\ 0 & \text{otherwise} \end{cases}$$

2. Tri-Cube kernel (compact, has 2 continuous derivatives at the boundary of its support)

$$D(t) = \begin{cases} (1 - |t|^3)^3 & \text{if } |t| \leq \lambda \\ 0 & \text{otherwise} \end{cases}$$

where $t = |x - x_0|$

3. Gaussian density function (continuously differentiable, but infinite support, could be more taxing on computation):

$$D(t) = \frac{\exp(-\frac{t^2}{2\sigma^2})}{\sqrt{2\pi}\sigma}$$

where σ is the "width"

Alternatively, we can also use **adaptive neighborhoods** where the λ parameter depends on the location x_0 For example,

$$K_\lambda(x_0, x) = D(\frac{|x - x_0|}{h_\lambda(x_0)})$$

Notes:

- Large λ implies lower variance (we average over more observations, larger n), but higher bias (we are approximating $f(x_0)$ with observation further away from x_0
- An adaptive parameter $h_\lambda(x)$ can keep the bias **constant** but the variance is **inversely proportional** to the local density of observation.
- Biggest problem with kernel smoothers is estimation near the boundaries (primarily due to the asymmetry of the kernels near the boundary); hence, the fits are badly biased near the boundaries, which Local Regression can help.

4.2 Selecting the Kernel Width

λ controls the "width" of kernel K_λ

- Epanechnikov/Tri-Cube: λ is the radius of the support region
- Gaussian: λ is the standard deviation
- Nearest Neighbor: λ is the number of k of nearest neighbors

Notes:

- Smaller width means less averaging, hence higher variance and smaller bias
- Larger width means more averaging, hence less variance and higher bias

To select λ, we typically use Cross-validation

4.3 Kernel Density Estimation

Another use of kernels is to estimate density (which was the original purpose of kernel before kernel regression).

Kernel density estimation is

Assume random sample $x_1, \ldots, x_n$ from a probability density $f_X(x)$ and we want to estimate f_X at a point x_0 (assuming one dimension $X \in R$)

The **Parzen estimate** is

$$\hat{f}_X(x_0) = \frac{1}{n\lambda} \sum_{i=1}^{n} K_\lambda(x_0, x_i)$$

where a choice of kernels is the Gaussian kernel with standard deviation λ, which could be extended to higher dimensional density estimation by using multivariate (Gaussian) kernels.

Non parametric density estimates can be used under the Bayes' theorem

Suppose J classes and fit the nonparametric densities $\hat{f}_j(X), j = 1, \ldots, J$ separately for each class

Assume we have prior probability for each class $\hat{\pi}_j$ (which might be from the sample proportions or some previous knowledge), then

$$\hat{P}(G = j|X = x_0) = \frac{\hat{\pi}_j \hat{f}_j(x_0)}{\sum_{k=1}^{J} \hat{\pi}_k \hat{f}_k(x_0)}$$

For more details, see Hastie et al. (2001) (section 6.6)

Chapter 5

Local Regression

- This method improve bias from Kernel Smoothing at the boundaries by fitting straight lines instead of constants.
- Also known as the **locally weighted linear regression**

It seeks to

$$\min_{\alpha(x_0),\beta(x_0)} \sum_{1}^{n} K_\lambda(x_0, x_i)[y_i - \alpha(x_0) - \beta(x_0)x_i]^2$$

and gives the estimate $\hat{f}(x_0) = \hat{\alpha}(x_0) + \hat{\beta}(x_0)x_0$

To fit this, we let $b(x)^T = (1, x)$ then define the $n \times 2$ matrix $\mathbf{B}$ to have i-th row given by $b(x_i)^T$

Then, let $\mathbf{W}(x_0)$ be the $n \times n$ diagonal matrix with the i-th diagonal given by $K_\lambda(x_0, x_i)$. Then

$$\hat{f}(x_0) = b(x_0)^T(\mathbf{B}^T\mathbf{W}(x_0)\mathbf{B})^{-1}\mathbf{B}^T\mathbf{W}(x_0)\mathbf{y} = \sum_{i=1}^{n} l_i(x_0)y_i$$

Thus, the fitted solution is linear in the y_i with the weights in the linear combination given by $l_i(x_0)$, which are a combination of the kernel weights and least squares equations, which is called **equivalent kernel**

An advantage of this method is that it corrects the boundary bias issues to first order (i.e., the bias is only a function of the second derivatives of $f(x_0)$ and so is small on average).

Thus, there is a small price we pay in that this fit tends to be a little bit biased in regions of curvature in the true function.

We can improve this bias by using **locally polynomial regression** (e.g., **locally quadratic regression**):

$$\min_{\alpha(x_0),\beta_j(x_0),j=1,\ldots,d} \sum_{i=1}^{n} K_\lambda(x_0,x_i)[y_i - \alpha(x_0) - \sum_{j=1}^{d} \beta_j(x_0)x_i^j]^2$$

which gives the solution $\hat{f}(x_0) = \hat{\alpha}(x_0) + \sum_{j=1}^{d} \hat{\beta}_j(x_0)x_0^j$

The reduction in bias under locally quadratic fit offset by higher variance

Notes:

- Local linear fits are much better at the boundaries than higher order polynomials in reducing bias, without leading to excessive variance increase.
- Local quadratic fits do not help bias much at the boundaries, but increase the variance a lot
- Local quadratic fits are usually used in reducing bias from curvature in the interior of the domain
- Local polynomials of odd degree dominate those of even degree (based on asymptotic analysis because asymptotically the MSE is dominated by boundary effects).

5.1 Higher Dimensions

To generalize Kernel Smoothing and Local Regression with p-dimensional kernel, we consider local hyperplanes

Example:

- If $d = 1$ (polynomial degree) and $p = 2$, we have $b^T(X) = (1, X_1, X_2)$
- If $d = 2$ and $p = 2$ we have $b^T(X) = (1, X_1, X_2, X_1^2, X_2^2, X_1X_2)$

then, we solve

$$\min_{\beta(x_0)} \sum_{i=1}^{n} K_\lambda(x_0,x_i)[y_i - b(x_i)^T\beta(x_0)]^2$$

to fit $\hat{f}(x_0) = b(x_0)^T\hat{\beta}(x_0)$

Then, the kernels of the general form would be

$$K_\lambda(x_0,x) = D(\frac{||x - x_0||}{\lambda})$$

where $||.||$ is the Euclidean norm.

Note: because the Euclidean norm is sensitive to the scale fo the variables, we have to standardize the predictors first.

The boundary bias issues are exponentiated under higher dimensions because of the Curse of dimensionality (larger fraction of the data locations are on or near the boundary)

Similar to the one-dimensional case, the local polynomial regression accounts for this boundary bias issue to first order. However, the problem is to balance localness (low bias) and low variance as the dimension increases without the sample size increasing exponentially in p

Hence, we do not see and **should not use smoothing or local polynomial regression in dimensions beyond three.**

Adding structural assumption to the modeling framework can help in cases where the dimension to sample size ratio high.

An example under kernels could be

$$K_{\lambda,\mathbf{A}(x_0,x)} = D(\frac{(x-x_0)^T\mathbf{A}(x-x_0)}{\lambda})$$

where $\mathbf{A}$ is a $p \times p$ positive semi-definite matrix, where it can give different weights to the elements of $x - x_0$ and/or recombine the elements fo this vector in various ways to shrink some dimensions ("compress") the points into a smaller region of space

Another way would be do add structure through **varying coefficient models**

Assume p inputs and a regression on the first q variables where the parameters depend on the remaining inputs $Z = (X_{q+1}, X_{q+2}, \dots, X_p)$

$$f(x) = \alpha(Z) + \beta_1(Z)X_1 + \cdots + \beta_q(Z)X_q$$

where the coefficients are functions of Z

Under Kernel weighted local regressions, then

$$\min_{\alpha(z_0),\beta(z_0)} \sum_{i=1}^{n} K_\lambda(z_0, z_i)[y_i - \alpha(z_0) - x_{1i}\beta_1(z_0) - \cdots - x_{qi}\beta_q(z_0)]^2$$

Chapter 6

Generalized Additive Models

- A more flexible method for nonlinear regression models

$$E(Y|X_1, X_2, \dots, X_p) = \alpha + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p)$$

where $f_j(.), j = 1, \dots, p$ are unspecified smooth (nonparametric) functions

Notes:

- We call this model additive because we separate f_j for each X_j
- Complicated functions for each features are allowed, and we can still retain the interpretability

Equivalently, the model can be rewritten as

$$y_i = \alpha + \sum_{j=1}^{p} f_j(x_{ij}) + \epsilon_i$$

ϵ usually is chosen to follow Gaussian distribution

- If each of the function $f_j(.)$ were written as a basis expansion, then we are back at least squares
- A general approach to fit each function would be a "scatterplot smoother" (e.g., cubic Smoothing splines or Kernel Smoothing) using an algorithm that simultaneously estimates all p-functions
- Transformation of the mean response (e.g., link function) as in Generalized linear models can also be applied.

Example:

Consider two-group classification via logistic regression. We relate the mean of the binary response

$$\mu(X) = P(Y = 1|X)$$

to functions of the predictors via an additive regression model using a logit link function

$$\log(\frac{\mu(X)}{1-\mu(X)}) = \alpha + f_1(X_1) + \cdots + f_p(X_p)$$

where each f_j is an unspecified smooth function

In general, we relate the conditional mean response to the additive function of the predictors via a link function, g:

$$g(\mu(X)) = \alpha + f_1(X_1) + \cdots + f_p(X_p)$$

Examples:

- $g(\mu) = \mu$ the identity link
- $g(\mu) = logit(\mu)$ or $g(\mu) = probit(\mu)$ as in the glm case for binary and binomial data (probit function is the inverse Gaussian cumulative distribution function: $probit(\mu) = \Phi^{-1}(\mu)$
- $g(\mu) = \log(\mu)$: Poisson count data

Function f_j can be nonlinear or linear or any other parametric forms. or a combination of them:

- $g(\mu) = X^T\beta + \alpha_k + f(Z)$: a semi-parametric model, where
  - X is a vector of predictors to be modeled linearly
  - α_k the effect for the k-th level of a qualitative input V
  - the effect of the predictor Z is modeled nonparametrically
- $g(\mu) = f(X) + g_k(Z)$ where
  - k indexes the levels of a qualitative input V and thus creates an interaction term with Z, $g(V, Z) = g_k(Z)$
- $g(\mu) = f(X) + g(Z, W)$: where g is a nonparametric function in two features, Z and W

6.1 Fitting Additive Models

We choose the function f_j to be "smoothers." Then, we minimize a penalized sum of squares

$$PRSS(\alpha, f_1, \dots, f_p) = \sum_{i=1}^{n}(y_i - \alpha - \sum_{j=1}^{p} f_j(x_{ij}))^2 + \sum_{j=1}^{p} \lambda_j \int f_j''(t_j)^2 dt_j$$

where the $\lambda_j \geq 0$ are tuning parameters

The minimizer is an additive cubic spline model with each of the functions f_j a cubic spline in X_j and with knots at each of the unique values of $x_{ij}, i = 1, \dots, n$

This solution is not unique because α is not identifiable without another constraint. Hence, we add $\sum_{i=1}^{n} f_j(x_{ij}) = 0 \forall j$ which implies $\hat{\alpha} = ave(y_i)$

- If $\mathbf{X} = \{x_{ij}\}$ is of full column rank, then the minimizer is unique
- If this isn't the case (i.e., $\mathbf{X}'\mathbf{X}$ is singular) then the **linear portions** of the components f_j c**annot be uniquely determined, but the nonlinear parts can**

To fit the model, we have to use an iterative (backfitting) procedure

Let S_j be the cubic smoothing spline for the j-th function. We apply it to the target residuals $\{y_i - \hat{\alpha} - \sum_{k \neq j} \hat{f}(x_{ik})\}_1^n$ as a function of x_{ij} to obtain the new estimate $\hat{f}_j$

The procedure is done for each predictor using the current estimates of the other functions $\hat{f}_k$ when computing the differences.

The process continue until the estimates $\hat{f}_j$ stabilize

6.1.1 The Backfitting Algorithm for Additive Models

Step 1: Initialize

$$\hat{\alpha} = \frac{1}{N}\sum_{1}^{N} y_i, \hat{f}_j \equiv 0, \forall i, j$$

Step 2: $j = 1, 2, \dots, p, \dots, 1, 2, \dots, p, \dots$

$$\hat{f}_j \leftarrow S_j[\{y_i - \hat{\alpha} - \sum_{k \neq j} \hat{f}_k(x_{ik})\}_1^N]$$

$$\hat{f}_j \leftarrow \hat{f}_j - \frac{1}{N}\sum_{i=1}^{N} \hat{f}_j(x_{ij})$$

until the functions $\hat{f}_j$ change less than a prespecified threshold

Notes:

- Any smoother can be used to take the place of S_j such as
  - other univariate regression smoothers (e.g., local polynomial regression and Kernel Smoothing)
  - linear regression operators yielding polynomial fits, piecewise constant fits, parametric spline fits, Fourier series fits
  - smoothers for second or higher-order interactions or periodic smoothers for seasonal effect.
- If we consider the smoother at the training points, it can be represented by the $n \times n$ smoother matrix $\mathbf{S}_j$ which can be used to estimate the df associated with the j-th term from $trace(\mathbf{S}_j) - 1$

6.2 Fitting Generalized Additive Models

- The optimization criterion here is the penalized log-likelihood
- It maximizes by combining the backfitting procedure with a likelihood maximizer
- The weighted linear regression component is replaced by a weighed backfitting algorithm Hastie et al. (2001) (section 9.2)

6.2.1 Local Scoring Algorithm

Step 1: Compute $\hat{\alpha} = \log[\frac{\bar{y}}{1-\bar{y}}]$ where $\bar{y} = ave(y_i)$, the sample proportion of ones, and set $\hat{f}_j \equiv 0 \forall j$

Step 2: Define $\hat{\eta}_i = \hat{\alpha} + \sum_j \hat{f}_j(x_{ij})$ and $\hat{p}_i = \frac{1}{1+\exp(-\hat{\eta}_i)}$

Iterate:

1. Construct the working target variable: $z_i = \hat{\eta}_i + \frac{y_i - \hat{p}_i}{\hat{p}_i(1-\hat{p}_i)}$
2. Construct weights $w_i = \hat{p}_i(1-\hat{p}_i)$
3. FIt an additive model to the targets z_i with weights w_i, using a weighted backfitting algorithm. This gives new estimates $\hat{\alpha}, \hat{f}_j, \forall j$

Step 3: Continue step 2, until the change in the functions falls below a prespecified threshold.

6.3 Summary

Pros	Cons
we can fit a nonlinear f_j to each X_j and automatically model nonlinear relationships (which we can't do under standard linear regression)	Model is restricted to be **additive**. Technically, we can include interactions, but we have to make them manually, and we can only do so with low-dimensional interaction terms as additive functions
The nonlinear fits can make more accurate **predictions**	Difficult to fit in high dimensions (because all variables have to be fit)
Since the model is additive, we can examine the effect of each X_j on Y individually while holding all of the other variables fixed (interprebility)	
The smoothness of the function f_j for the variable X_j can be summarized by df	

An extension of GAM is to include random effects, which is called **Generalized Additive Mixed Models** (GAMMs) (can refer to this book)

6.4 Application

To overcome Spline Regression's requirements for specifying the knots, we can use Generalized Additive Models or GAM.

```
library(mgcv)
```

```
## Warning: package 'mgcv' was built under R version 4.0.5
```

```
## Loading required package: nlme
```

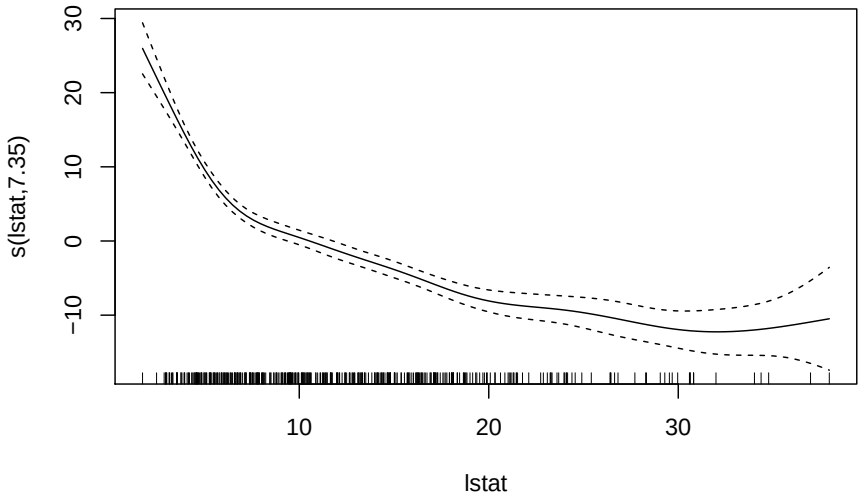
```
## Warning: package 'nlme' was built under R version 4.0.5
```

```
##
## Attaching package: 'nlme'
```

```
## The following object is masked from 'package:dplyr':
##
##     collapse
```

```
## This is mgcv 1.8-36. For overview type 'help("mgcv-package")'.
```

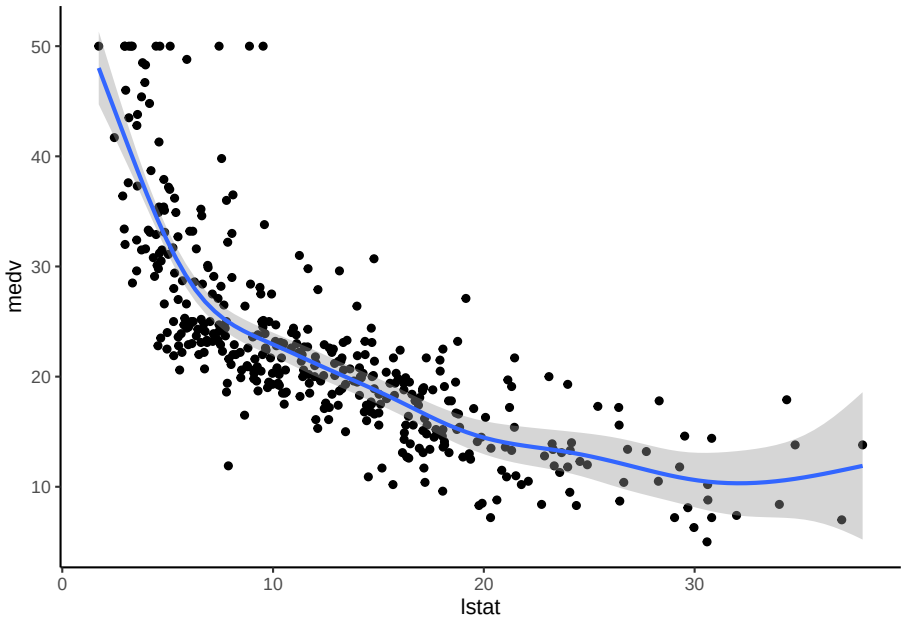
```
# Build the model
model <- gam(medv ~ s(lstat), data = train.data)
plot(model)
```




```
# Make predictions
# predictions <- model %>% predict(test.data)
# Model performance
data.frame(
  RMSE = RMSE(predictions, test.data$medv),
  R2 = R2(predictions, test.data$medv)
)
```

```
##       RMSE        R2
## 1 5.317372 0.6786367
```

```
ggplot(train.data, aes(lstat, medv) ) +
  geom_point() +
  stat_smooth(method = gam, formula = y ~ s(x))
```




```
detach(train.data)
```

Chapter 7

Gaussian Process Regression

Suggested reading:

- Rasmussen and Williams (2005)
- Wikle et al. (2019) (section 4.2)

Under Gaussian Process (GP), we assume that Y is a random (Gaussian) process that is controlled by a mean function and covariance function.

Response model:

$$z_i = Y_i + \epsilon_i, i = 1, \dots, n$$

Assumptions:

- Y = a latent process specified by a GP
- ϵ_i independent and identically distributed mean-zero observation errors that are independent of Y and have variance σ^2_ϵ
- the errors are assumed to have a normal (Gaussian) distribution $\epsilon_i \sim iidN(0, \sigma^2_\epsilon)$ these errors could be correlated

Consider a **linear predictor** of the form

$$\hat{Y}(x_0) = k_0 + \sum_{i=1}^{n} w_i z_i$$

where w_i are weights and k_0 is some constant

To pick these weights, we choose "optimal" weight based on mean squared error notion, which requires us to consider the random process that controls Y

7.1 Latent Process Model

Assume that we have the latent process follows the model

$$Y(x) = f(x)$$

where $f(x) \sim GP(m(x), c(x, x'))$ is a random process (specifically a Gaussian Process) and

- $m(x)$ = mean function $E[f(x)]$
- $c(x, x')$ = covariance function $c(x, x)' = E[(f(x) - m(x))(f(x') - m(x'))^T]$

The difference between **Gaussian distribution** and a **Gaussian process** is

- Gaussian Distribution: a distribution over vectors
  - Fully specified by a **mean vector** and **covariance matrix**: (e.g., $\mathbf{Y} \sim N(\ ,\ )$ which is a multivariate normal distribution)
- Gaussian Process: a distribution over functions
  - Fully specified by a mean function (m) and covariance function (c) (e.g., $Y \sim GP(m, c)$, where we specify a mean function and covariance function to define a GP)

7.2 Gaussian Processes

- A GP is an infinite-dimensional object
- Allow us to make a prediction anywhere in our x domain assuming that we know the mean and covariance functions (data generating process), we need the covariance between ANY two x locations
- We actually need to consider finite-dimensional distributions because
  - We only have a finite set of data and also we are interested in a finite set of predictions
  - Any finite collection of GP random variables has a joint Gaussian distributions (i.e., a multivariate normal distribution).

7.3 Gaussian Distributions

$$\begin{pmatrix} \mathbf{v} \\ \mathbf{v}_* \end{pmatrix} \sim Gau\left(\begin{pmatrix} \\ _* \end{pmatrix}, \begin{pmatrix} & ^* \\ ^T & ^{**} \end{pmatrix}\right)$$

where $E(\mathbf{v}) = \ , E(\mathbf{v}_*) = \ _*, \ = cov(\mathbf{v}, \mathbf{v}), \ = cov(\mathbf{v}, \mathbf{v}_*), \ _{**} = cov(\mathbf{v}_*, \mathbf{v}_*)$

Then, partition

$$\mathbf{v}_*|\mathbf{v} \sim Gau(\ _* + \ _*^{T} \ ^{-1}(\mathbf{v} - \ ), \ _{**} - \ _*^{T} \ ^{-1} \ _*)$$

7.4 Optimal Linear Prediction

We are interested in predicting $Y(x_0)$ (i.e., at some arbitrary x) given the data $\mathbf{z} \equiv (z_1, ..., z_n)^T$

Assume that the mean function $\mu(x) \forall x$ (e.g., a constant)

We seek linear predictors that minimize the mean squared prediction error of the form

$$\begin{aligned} \hat{Y}(x_0) &= k(x_0) + \sum_{i=1}^{n} w_i(x_0) z_i \\ &= k(x_0) + \mathbf{w}_0^T \mathbf{z} \end{aligned}$$

The optimal linear predictor is also the conditional expectation $E(Y(x_0)|\mathbf{z})$.

Hence, we seek the conditional distribution $Y(x_0)|\mathbf{z}$

Rewriting the data model in vector form:

$$\mathbf{z} = \mathbf{Y} + \epsilon, \epsilon \sim Gau(\mathbf{0}, \mathbf{C}_\epsilon)$$

where $\mathbf{Y}$ and ϵ are ordered in the same way as the data vector $\mathbf{z}$

$$cov(\mathbf{Y}, \mathbf{Y}) \equiv \mathbf{C}_y$$

$$cov(\epsilon, \epsilon) \equiv \mathbf{C}_\epsilon$$

$$cov(\mathbf{z}, \mathbf{z}) = \mathbf{C}_y + \mathbf{C}_\epsilon$$

$$\mathbf{c}_0^T \equiv cov(Y(x_0), \mathbf{z})$$

$$c_{0,0} \equiv var(Y(x_0))$$

μ_0 is mean at x_0 and is the mean corresponding to the observations.

Then, the joint Gaussian distribution:

$$\begin{pmatrix} Y(x_0) \\ \mathbf{v}_* \end{pmatrix} \sim Gau\left(\begin{pmatrix} \quad \end{pmatrix}, \begin{pmatrix} c_{0,0} & \mathbf{c}_0^T \\ \mathbf{c}_0 & \mathbf{C}_z \end{pmatrix}\right)$$

Using the joint/conditional Gaussian Distributions formula

$$Y(x_0)|\mathbf{z} \sim Gau(\mu_0 + \mathbf{c}_0^T\mathbf{C}_z^{-1}(\mathbf{z} - \ ), c_{0,0} - \mathbf{c}_0^T\mathbf{C}_z^{-1}\mathbf{c}_0)$$

The predictor is given by the **mean** of this distribution

$$\hat{Y}(x_0) = \mu_0 + \mathbf{c}_0^T\mathbf{C}_z^{-1}(\mathbf{z} - \ )$$

and the prediction **variance** is

$$\sigma^2_{\hat{Y}_0} = c_{0,0} - \mathbf{c}_0^T\mathbf{C}_z^{-1}\mathbf{c}_0$$

This predictor is the **optimal predictor** because it minimizes the **mean squared prediction error**:

$$MSPE = E[(Y(x_0) - \hat{Y}(x_0))^2]$$

Hence, the **weights** are given by

$$\mathbf{w}_0 = \mathbf{c}_0'\mathbf{C}_z^{-1}$$

and

$$k(x_0) = \mu_0 - \mathbf{c}_0'\mathbf{C}_z^{-1}$$

which is an effective kernel (just like Spline Regression and Local Regression

These formulas can be extended to multiple locations (each location has its own vector of weights). However, if there are a lot of sample locations, there could be computational problem regarding calculating the matrix inverse $\mathbf{C}_z^{-1}$

In cases where you have no measurement errors (i.e., $z_i = y_i$) (equivalently, replacing $\mathbf{C}_z$ with $\mathbf{C}_y$)

7.5 Covariance Functions

To predict at new x locations, we must specify:

1. a mean function (doable)
2. a covariance function between ANY two locations (hard)

In the real world, we hardly ever know the variances and covariances that make up $\mathbf{C}_y, \mathbf{C}_\epsilon, \mathbf{c}_0, c_{0,0}$

A seemingly simple solution is to specify covariance functions in terms of parameters that can give a covariance between any pairs of coordinates and then estimate the parameters, which is the same thing that we did in Linear Mixed Models (e.g., AR models, exponential models)

- **Covariance functions must be positive semi-definite** to ensure that prediction variances are non-negative
- Covariance functions should be realistic (i.e., account for realistic dependencies), which can be difficult sometime. Typically, the assumption that "nearby" observations (similar in the case of Kernel Smoothing) are more dependent is sufficient.

In practice, we usually have to employ a **stationarity assumption**

Example:

If the random process is assumed to be second-order (weakly) stationary then it has constant expectation and a covariance function that can be expressed in terms of the distance between the x locations ($h = x - x'$), we specify

$$c(x, x') = c(h, \ )$$

In two or more dimensions, if this distance lag does not depend on direction (i.e., $h = ||\mathbf{h}||$), then we call the function **isotropic**

The benefits of stationarity assumption:

- allows for more **parsimonious parameterization** of the covariance function
- provides **pseudo-replication** of dependencies at given lags, which facilitate estimation

To construct covariance functions with this assumption, traditional machine learning would use **separable covariance functions** (e.g., for x_1 and x_2 - 2 dimension prediction)

$$c(h_1; h_2) \equiv c(h_1) \times c_2(h_2)$$

which is valid if both covariance functions are valid

This separability property helps with computation in prediction locations (e.g., matrix inverses we mention above), but it can also be unrealistic in cases like spatio-temporal data

Then to have valid spatial and temporal covariance functions, we can employ the following covariance functions:

- The Matern
- Power exponential: $c(\tau, \theta) = \sigma^2 \exp\{-\frac{|\tau|}{\theta}\}$ where σ^2 is the variance parameter and θ is the dependence parameter
- Gaussian classes

To estimate the parameters in the covariance functions, and in the mean function we can either

1. Under the frequentist perspective, we can use REML, MLE (like Linear Mixed Models) and BLUP for random process
2. Under the Bayesian perspective, we use GP for $f(x)$ as prior distribution, and update this with the observations, $\mathbf{z}$

With a lot of data locations, REML, MLE, and Bayesian can be problematic because of the high-dimensional covariance matrix $\mathbf{C}_z, \mathbf{C}_y$ (which research are still being conducted to improve)

Notes:

- If we model the mean function in terms of covariates ($\mu(x) = \mathbf{w}(x)'$ m where $\mathbf{w}$ is a vector of covariates. Then, it's Linear Mixed Models with difference that the covariates must be available for ANY x
- In cases where you don't measurement uncertainty in the observation, (ϵ) then the prediction at an observation location will be exactly the same as the observation. And the other way around will filter the uncertainty with the observation
- we can also have non-Gaussian observations conditioned on the GP (like GLMM)

Chapter 8

Classification and Regression Trees

- Segmenting the predictor space into a number of simple regions.
- Develop a collection of splitting rules that make these segments.
- The splitting rules can be thought of as "tree" (therefore, "decision trees")
- Decision trees can be used for both classification and regression, ergo the name CART
  - Under regression, the prediction is the average of the partitioned predictor space (i.e., segments)
  - Under classification, we classify based on the most prevalent class in each segment
- Compared to linear models
  - If the underlying relationship is linear, the linear regression/ classification setting works better than CART
  - IF the underlying relationship is nonlinear and complex between the features and the response, the CART model can be better.
- Issues:
  - Categorical predictors can prove to be difficult to implement CART when we have $q > 2$ possible unordered values of the predictors as there are $2^{q-1} - 1$ possible partitions of the q values into 2 groups. Usually lead to over-fitting.
  - Missing predictor values:

    - ∗ For categorical variables, we can make a separate category for "missing". Or we can construct surrogate variables that mimic the split of the training data using only the non-missing variables
  - Non-binary splits: not recommended because it splits the data too quickly and doesn't leave enough data for the next level of the tree
  - Linear Combination Splits: we can make linear combination of the variables to choose the splits (akin to PC-regression), which can help prediction, but reduce interpretation
  - Instability of trees: CART has high variance (i.e., a small change in the data can lead to different splits). To handle this problem, we can use "bagging" (i.e., bootstrapping procedure). This can happen even after you prune.
  - Lack of Smoothness: CART can lead to non-smooth prediction surface, which can be problematic for regression applications. The MARS procedure can be used instead
  - Additive structure: Since CART uses binary structure, it can't accommodate additive structure, but MARS can
  - Trees do not have good predictive accuracy as other regression and classification approaches.

Advantages:

- Easily explainable (to non-statisticians)
- Decision trees may be closer to human decision making than traditional regression and classification
- Trees are easily interpreted (with graphical depiction)
- Trees can handle qualitative predictors without dummy variables

8.1 Regression Trees

Say we have p inputs and a response for each of n observations, $(x_i, y_i), i = 1, \ldots, n$ with $x_i = (x_{i1}, \ldots, x_{ip})^T$

Suppose that we have a partition of the predictor space into J regions $R_1, \ldots, R_j$. Then, our prediction in each region is some constant, c_j. Thus, we could write the response function as

$$f(x) = \sum_{j=1}^{J} c_j I(x \in R_j)$$

where $I(x \in R_j)$ is an indicator function (1 when x is in the j-th region, 0 otherwise).

Given the partitions, the estimate $\hat{c}_j$ is the average of of y_i in R_j

$$\hat{c}_j = \hat{y}_{R_j} \equiv ave(y_i | x_i \in R_j)$$

To find the partitions, we seek to divide the predictor space into high-dimensional rectangles (boxes) (i.e., we want to find the boxes $R_1, ..., R_j$ that minimize the residual sum of squares)

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

where $\hat{y}_{R_j}$ is the mean response for the training observations within the j-th box (e..g, $\hat{c}_j$)

Since we can't consider every possible partition (computationally constraint), we seek binary partitions and specifically, a top down, greedy approach (i.e., at each step the best split is made at that step, once we make a split, we don't go back and change it) known as **recursive binary splitting**.

8.1.1 Recursive Binary Splitting

Step 1: we select the predictors X_j and the cutpoints such that splitting the predictor space into the regions $\{X|X_j < s\}$ and $\{X|X_j \geq s\}$ leads to the maximum reduction in RSS (i.e., for any j and s, we define the regions (half-planes):

$$R_1(j, s) = \{X|X_j < s\}; R_2(j, s) = \{X|X_j \geq s\}$$

and seek the value of j and s the minimizes

$$\sum_{i:x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

where $\hat{y}_{R_1} = \hat{c}_1$ is the mean response in $R_1(j, s)$

Notes:

- When p is small, it's computationally efficient to find values of j and s.
- After the partition is selected, we repeat the process, looking for the best predictor (j) and best cut point (s) to split one of the two new regions to minimize the RSS, which will give us three regions. We continue to split one of these three regions to minimize the RSS

- The stopping criterion is usually when a region contains fewer than some specified number of observations (because we have to the averages of these regressions for our mean response)
- The end regressions of the tree $(R_1, ..., R_J)$ are known as **terminal nodes** or **leaves** of the tree
- The points along the tree where the predictor space is spit are **internal nodes**
- Regression trees can usulaly lead ot over-fitting ont he test set, to reduce this problem, we can prune (i.e., remove leaves) to obtain a **subtree** using cross-validation.

8.1.2 Cost complexity pruning

- laso knwon as **weakest link pruning**

Wefirst gorw a verg large tree T_0. Instread of considering eveyr psosible subtree, we consider a sequnece of trees indeed a non-negative tuning paratmers, α (i.e., for each value of α there is a subtree T such that $T \in T_0$ that minimizes

$$\sum_{j=1}^{|T|} \sum_{i:x_i \in R_j} (y_i - \hat{y}_{R_j})^2 + \alpha |T|$$

where $|T|$ is the number of terminal nodes

The parameter α controls a tradoff between the subtree's complexity and how well it fits the training data ($\alpha = 0$ implies $T = T_0$). As α gets larger, the penalty for a larger tree is higher, so it penalizes similarly to the Lasso regression. Moreover, as α increases, the branches get pruned in a nested a predictable fashion. Hence, it's possible to get the whole sequence of subtrees as a function of α (based on cross-validation). After getting α using cross-validation, we can use that value with the full data set to obtain the fitted subtree

8.1.3 Algorithm for regression tree

1. Recursive Binary Splitting for growing a tree with your choice of stopping rule (e.g., observation-based)
2. Use Cost complexity pruning to obtain a sequence of best subtrees as a function of α
3. Use K-Fold Cross-Validation to choose α

8.2 Classification Trees

When we have a qualitative response we can use classification tree.

The main difference from Regression Trees is that we predict the response based on the most commonly occurring class in the region associated with that terminal node.

We may also be interested in the class proportion for each node when interpreting a classification tree

We grow a classification tree in the same manner as the Regression Trees, with Recursive Binary Splitting. However, instead of using RSS we use other measure related to the classification summary.

First we define the proportion of class k observation in the j-th node as

$$\hat{p}_{jk} = \frac{1}{n_j} \sum_{x_i \in R_j} I(y_i = k)$$

we classify an observation in node j to class k with the maximum value of $\hat{p}_{jk}$

The three measures for growing tree:

1. Classification error rate: $1 - \max_k(\hat{p}_{jk})$
    - measures the fraction of the training observations in that region that do not belong to the most common class
2. Gini index: $\sum_{k=1}^{K} \hat{p}_{jk}(1 - \hat{p}_{jk})$
    - measures the total variance across the K classes
3. Cross-Entropy or Deviance: $-\sum_{k=1}^{K} \hat{p}_{jk} \log(\hat{p}_{jk})$
    - related to an information measures and the deviance

The Gini index and Cross-Entory both take small values when all $\hat{p}_{jk}$ are close to 1 or 0. Hence, we say they measure node purity, which are more sensitive to node probabilities than the classification error rate. Hence, we typically use the last two to grow the tree.

Rule of thumb: use Gini and Cross-Entropy to build and prune a tree and use classification error rate for pruning if prediction accuracy of the pruned tree is the goal

Chapter 9

Multivariate Adaptive Regression Splines

- Also known as MARS
- an adaptive procedure for regression when there are a large number of input variables.
- Can also be viewed as a generalization of stepwise linear regression or a modification of the CART procedure that improves its performance in the regression setting
- uses expansions in the piecewise linear [basis function][Basis Function Representations]
- To mimic CART, the MARS procedure makes the following changes:
  - Replace the piece wise linear basis function by step functions (e..g, $I(x-t>0); I(x-t) \leq 0)$
  - When a model term is in a multiplication by a candidate term, it gets replaced by the interaction, and hence it not available for further interaction
  - With these changes, the MARS forward procedure is similar to the CART tree-growing algorithm (i.e., multiplying a step function by a pair of reflected step functions is equivalent to splitting a node at the step. The second restriction is a node may not be split more than once (leaning to the binary tree structure)). Hence, MARS is more powerful because it allows for additive structures because it's not limited to the binary tree structure
- MARS is not used that much today anymore.

Consider basis function of the form ("+" means "positive part" with a not at t)

$$(x-t)_+ \begin{cases} x-t & \text{if } x > t \\ 0 & \text{otherwise} \end{cases}$$

and

$$(t-x)_+ \begin{cases} t-x & \text{if } x < t \\ 0 & \text{otherwise} \end{cases}$$

These are linear splines and the pair is **reflected pair**.

We form such reflected pairs for each input X_j with knots at each observed value x_{ij} of that input variable.

The collection of basis function is

$$C = \{(X_j - t)_+, (t - X_j)_+\}_{t \in \{x_{1j}, \ldots, x_{nj}\}}, j = 1, \ldots, p$$

there can be as many as $2np$ basis functions (if all of the input values are distinct), which can be large.

Consider a model fo the form

$$f(X) = \beta_0 + \sum_{m=1}^{M} \beta_m h_m(X)$$

where each $h_m(X)$ is a function in C, or a product of two or more such functions.

Similar to forward stepwise linear regression, we try to build a final model using these functions of the inputs (instead of the original inputs)

For a given choice of h_m, we estimate the β_m using linear squares (i.e., minimizing the residual sum of squares). Hence, the choice of $h_m(X)$ is very important

First, we choose the constant function $h_0(X) = 1$ and all of the functions in C are candidates.

At each stage, we consider a new basis function pair all products of a function h_m in the existing model set M with one of the reflected pairs in C, i.e., the

$$\hat{\beta}_{M+1} h_l(X) \times (X_j - t)_+ + \hat{\beta}_{M+2} h_l(X) \times (t - X_j)_+, h_l \in M$$

that produces the largest decrease in training error

h_l here is some basis function that is already in the model

Then, these are added to the model set M

Example:

First, consider adding to the model a function fo the form $\beta_1(X_j - t)_+ + \beta_2(t - X)j)_+ : t \in \{x_{ij}\}$ (multiplication by the constant function produces the function itself)

Assume that for a particular data set the best choice is $\hat{\beta}_1(X_2 - x_{72})_+ + \hat{\beta}_2(x_{72} - X_2)$ we then add this to the set M

Then, we consider adding a pair of products of the form

$$h_m(X) \times (X_j - t)_+$$

$$h_m(X) \times (t - X_j)_+, t \in \{x_{ij}\}$$

where for h_m we have the choices

$$h_0(X) = 1, h_1(X) = (X_2 - x_{72})_+$$

or

$$h_2(X) = (x_{72} - X_2)_+$$

For example, the third choice produces functions such as $(X_1 - x_{51})_+ \times (x_{72} - X_2)_+$

The procedure continues until a pre-specified number of terms is in the model, which will be a large model that overfits the data.

Hence, we use backward deletion (i.e., the term that leads to the smallest increase in residual squared error is deleted at each stage). Thus, we have an estimated "best" model $\hat{f}_\lambda$ of each size (number of terms) λ

We could use cross-validation to choose the best model, but it's computationally intensive. Instead, we use generalized cross-validation, which is defined as

$$GCV(\lambda) = \frac{\sum_{i=1}^n (y_i - \hat{f}_\lambda(x_i))^2}{(1 - M(\lambda)/n)^2}$$

where $M(\lambda)$ is the effective number of parameters in the model. (i.e., accounting for λ and the number of parameters used in selecting the optimal position fo the knots).

Typically, we use $M(\lambda) = r + cK$ where

- r = number of linearly independent basis functions
- K = number of knots selected in the forward process
- $c = 3$ (typically) or 2 if the model is restricted to be additive

Advantages

- A major advantage of these piecewise linear basis functions is that they are "local" (i.e., non-zero in only a small portion of the features space, even when multiplied together), which facilitates model fitting in high dimensions.
- A computational advantage because it takes on the order of n operations to try very knot in each step of the process (for each of the $j = 1, \ldots, p$ variables)
- If higher order products are in the model, lower order components must also be included in the model. (in reality, we don't have to, but it's a good modeling practice)
- MARS can be extended to classification settings (e..g, setting the response variable to 0 and 1) but it's artificial. There are specific MARS-like algorithms designed for classification

Disadvantages

- Each input can occur at most one time in a product. Thus, **higher order powers of an input are not allowed**. If you include it, then it can produce unstable fits, particularly at the boundaries
  - One can also restrict products to be of a certain order (e..g, only second-degree or third-degree)

Chapter 10

Bagging

- Bootstrapping can help tree methods (e.g., CART)
- Since CART and MARS suffer from high variance (because if we split the training data into 2 samples at random and fit both halves, the results can be quite different), we use **bootstrap aggregation** (i.e., **bagging**), which is a general-propose procedure to reduce variance
- If the procedure has low variance, it will yield similar results if applied repeatedly to distinct data sets (e.g., as in linear regression when n is larger relative to p)
- The idea of bagging is that **averaging reduces variance** (somewhat like measurement error). To reduce variation, we take many training sets, and build a separate prediction model, and average them.

To execute bagging, we calculate $\hat{f}^1(X), \hat{f}^2(X), ..., \hat{f}^B(X)$ using B separate training set, and average them

$$\hat{f}_{bag}(X) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x)$$

However, in reality, we don't have access to multiple training sets, so we use bootstrap samples from our (single) training set (i.e., we generate B bootstrap sample - **sampling randomly with replacement**). Hence, $\hat{f}_{bag}(x)$ is the result of averaging the fits from these B bootstrap samples.

In the case of regression trees with bagging, we construct B regression trees and average the resulting predictions, but the trees are grown large with no pruning. Hence, each tree has high variance but low bias and averaging then reduces the variance. In practice, **100** samples is sufficient to reduce the variance significantly.

Notes:

- Bagging can be used for classification, where we record the class predicted by each of the B trees for a given test observation and then choose the category that occurs the most often.
- We can also estimate the error associated with a bagged model **without having to perform cross-validation**.
  - The observations not used in the bootstrap samples (for each sample) are the out-of-bag (OOB) observations.
  - We predict the response for the i-th observation using each of the trees in which the observation was OOB. Then, we average to get a single prediction, and get OOB MSE (or classification error)
  - OOB error is a valid estimate of the test error for the bagged model because the response for each observation is predicted using only the trees that were fit without using that observation.

Disadvantages

- Although bagging can improve predictive power (relative to a single tree), it reduced interpretation (i.e., when we average over many different trees, the variable splits (or basis function included) can change dramatically, preventing interpretation). Thus, we can't know which variable is important
  - To deal with this problem, we can keep track of (to get measure of variable importance in the tree)
    * the total amount that the residual sum of squares is decreased due to splits over a given predictor averaged over all B trees (in the case of regression), or
    * add up the total amount that the Gini index is decreased by splits over a given predictor, averaged over all B trees (in case of classification).

Part II

UNSUPERVISED LEARNING

Chapter 11

Boosting

- Similar to Bagging but more powerful, where we can improve the predictions from decision trees
- It was designed for classification problems (but now we can also apply to regression settings).
- In the machine learning world, this method combines the output of many "weak" learners to produce a power "committee"
- A classic boosting algorithm for classification is "AdaBoost.M1" (Freund and Schapire, 1997)
- It can be considered as one of the best off-the-shelf classifiers
- Under boosting, trees are *not grown independently*, but rather **sequentially**
- There are no bootstrap samples with boosting because we build trees on the residuals in a step-by-step manner
- In the literature, "fitting the data hard" means we fit a single large decision tree to the data, but we will **learn slowly** by fitting a decision tree to the residuals (not Y) from the model, which helps to improve the part of the tree that did not perform well in the existing tree structure.
- We add this new decision tree into the fitted function with a shrinkage parameter (λ) to slow the process down to add more trees to areas that don't fit well (also helps prevent overfitting). Hence, each tree after will depend strongly on the trees that have already been grown.

11.1 Regression Boosting

Algorithm for boosting regression trees

1. Set $\hat{f}(x) = 0$ and $r_i = y_i \forall i$ (in the training set)
2. For $b = 1, \ldots, B$ repeat
    1. Fit a tree $\hat{f}^b$ with d splits ($d+1$ terminal nodes) to the training data (X, r)
    2. Update $\hat{f}$ by adding in a shrunken version of the new tree $\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x)$
    3. Update the residuals $r_i \leftarrow r_i - \lambda \hat{f}^b(x_i)$
3. Final boosted model $\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x)$

Boosting has 3 tuning parameters:

1. B: the number of trees. boosting can overfit if B is too large, we can use Cross-validation Fitting to select B
2. λ: the shrinkage parameter, which controls the rate at which boosting learns. Typical values are 0.01 or 0.001 (best choice depends on the data at hand). The smaller the λ, the larger B must be to achieve good performance
3. d: the number of splits in each tree. $d = 1$ works well (surprisingly), which we call the tree a "stump." With $d = 1$, the boosted ensemble is fitting an additive model since each term only involves one variable. More generally, d is the interaction depth and controls the interaction order of the boosted model (since d splits can involve at most d variables).

11.2 Classification Boosting

For classification problem, consider a two-class classification code such that $Y \in \{-1, 1\}$ and a vector of predictor variables, X.

We have a classifier $G(X)$ that produces prediction that selects either -1 or 1. the error rate on the training sample is

$$\bar{err} = \frac{1}{n} \sum_{i=1}^{n} I(y_i \neq G(x_i))$$

A weak classifier is one whose error rate is only slightly better than random guessing. Now, boosting sequentially applies a weak classification algorithm to produce a sequence of weak classifiers, $G_m(x), m = 1, \ldots, M$ (M is above B)

The predictions are then combined through a **weighted majority vote** to produce the final prediction

$$G(x) = sign(\sum_{m=1}^{M} \alpha_m G_m(x))$$

where the weights $\alpha_m, m = 1, \ldots, M$ are computed by the boosting algorithm and give highest influence to the best classifiers

At each boosting step, weights $w_1, \ldots, w_n$ are applied to the training observations $(x_i, y_i), i = 1, \ldots, n$.

To start, weights are set equal, $w_i = \frac{1}{n}$ so there is no preference for the first classification.

For each successive iteration, the observation weights at iteration m are modified such that those observations that were misclassified by the classifier $G_{m-1}(x)$ have their weights increased, those that were correctly classified have their weights decreased

As iterations continues, observations that are difficult to classify correctly receive ever-increasing influence. Each successive classifier is forced to concentrate on those training observations that are missed by the previous classifiers in the sequence

AdaBoost.M1

1. Initialize the observation weights $w_i = \frac{1}{N}, i = 1, \ldots, N$
2. For $m = 1$ to M
    1. Fit a clisifer $G_m(x)$ to the training data with weights w_i
    2. Calculate $err_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i}$
    3. Calculate $\alpha_m = \log(\frac{1-err_m}{err_m})$
    4. Set $w_i \leftarrow w_i \times \exp(\alpha_m \times I(y_i \neq G_m(x_i))), i = 1, \ldots, N$
3. $G(x) = sign(\sum_{m=1}^{M} \alpha_m G_m(x))$

Why boosting works so well?

From the perspective of a basis function expansion (3.1:

$$f(x) = \sum_{m=1}^{M} \beta_m b(x; \gamma_m)$$

where

- β_m are the expansion coefficients
- $b(x, \gamma)$ are simple functions of the multivariate inputs x and parameters γ

then, boosting is a way of fitting such an additive expansion where the basis functions are the individual classifiers $G_m(x) \in \{-1, 1\}$

To fit this model, we minimize some loss function averaged over the training data (e..g, squared error loss).

$$\min_{\{\beta_m,\gamma_m\}_1^M} \sum_{i=1}^{n} L(y_i, \sum_{m=1}^{M} \beta_m b(x_i;\gamma_m))$$

For most loss functions $L(y, f(x))$ and basis functions $b(x;\gamma)$, the numerical optimization algorithms are usually computationally intensive.

In some cases, a solution is to solve the subproblem of fitting just a single basis function

$$\min_{\{\beta,\gamma\}} \sum_{i=1}^{n} L(y_i, \beta b(x_i;\gamma))$$

Then, we approximate the more general minimization problem by sequentially adding new basis functions to the expansion without adjusting the parameters and coefficients that have already been added. This process is called **forward stagewise additive modeling** (i.e., at each iteration m, we solve for the optimal basis function $b(x,\gamma_m)$ and coefficient β_m to add to the current expansion $f_{m-1}(x)$, which gives $f_m(x)$ and we repeat the process while previously added terms remain constant)

Example: squared error loss

$$L(y, f(x)) = (y - f(x))^2$$

then

$$\begin{aligned} L(y_i, f_{m-1}(x_i) + \beta_m b(x_i;\gamma_m) &= (y_i - f_{m-1}(x_i) - \beta_m b(x_i;\gamma_m))^2 \\ &= (r_{im} - \beta_m b(x_i;\gamma_m))^2 \end{aligned}$$

where $r_{im} = y_i - f_{m-1}(x)$ is the residual of the current model on the i-th observation.

The term $\beta_m b(x;\gamma_m)$ that best fits the current residuals is added to the expansion at each step

AdaBoost.M1 is equivalent to forward stagewise modeling using the loss function (Hastie et al., 2001, 10.4)

$$L(y, f(x)) = \exp(-y f(x))$$

which is the "exponential loss" (closely related to the binomial negative log-likelihood (deviance), although they have different properties in finite data (real-world) settings)

This loss function is attractive because it facilitates computation

There are more loss functions. Typically we might choose a loss function because

- it's robust to certain properties (e.g., outliers)
- it facilitates computation

When a loss function is differentiable, we can make use of the gradient of the loss function to improve the optimization, which is the basis of gradient boosting ("gradient boosting models (GBM)"; formerly, Multiple Additive Regression Trees (MART)).

11.3 Gradient Boosting

Part III

REINFORCEMENT LEARNING

Chapter 12

Interpretable Machine Learning

https://christophm.github.io/interpretable-ml-book/ https://en.wikipedia.org/wiki/Explainable_artificial_intelligence https://arxiv.org/pdf/1905.08883.pdf https://arxiv.org/abs/2004.14545 https://www.kaggle.com/learn/machine-learning-explainability https://towardsdatascience.com/interperable-vs-explainable-machine-learning-1fa525e12f48

TCAV:

- https://github.com/tensorflow/tcav
- http://proceedings.mlr.press/v80/kim18d.html
- https://arxiv.org/abs/1711.11279

PAIR:

- https://pair-code.github.io/facets/
- https://research.google/teams/brain/pair/

Embedding Project:

- https://projector.tensorflow.org/

What-If tools:

- https://pair-code.github.io/what-if-tool/
- https://cloud.google.com/ai-platform/prediction/docs/using-what-if-tool

12.1 SHAP

https://github.com/slundberg/shap https://meichenlu.com/2018-11-10-SHAP-explainable-machine-learning/ https://bjlkeng.github.io/posts/model-explanability-with-shapley-additive-explanations-shap/

12.2 Video Classification

https://hackernoon.com/continuous-video-classification-with-tensorflow-inception-and-recurrent-nets-250ba9ff6b85 https://cloud.google.com/video-intelligence https://arxiv.org/pdf/1609.06782.pdf https://www.tubefilter.com/2019/02/12/taxonomy-youtube-videos-develop-original-content-that-works/ https://medium.com/swlh/converting-news-into-video-stories-5d8d4fc5a32b https://towardsdatascience.com/introduction-to-video-classification-6c6acbc57356 https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0228579

12.3 Image Memorability

(AMNET)

https://www.researchgate.net/publication/335603628_Image_Memorability_Using_Diverse_Visual_Features_and_Soft_Attention https://www.researchgate.net/publication/308812354_Deep_Learning_for_Image_Memorability_Prediction_the_Emotional_Bias https://arxiv.org/pdf/2005.02969.pdf https://github.com/dazcona/memorability https://arxiv.org/abs/1804.03115 https://www.researchgate.net/publication/324387176_AMNet_Memorability_Estimation_with_Attention http://ceur-ws.org/Vol-2283/MediaEval_18_paper_24.pdf https://jov.arvojournals.org/article.aspx?articleid=2771173 https://openaccess.thecvf.com/content_cvpr_2018/papers/Fajtl_AMNet_Memorability_Estimation_CVPR_2018_paper.pdf https://ieeexplore.ieee.org/document/8578764

https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=7382237&tag=1 https://people.umass.edu/~yangzhou/

http://people.csail.mit.edu/khosla/projects/regionmem/ https://github.com/adikhosla/feature-extraction https://expertprogrammanagement.com/2019/12/elaboration-likelihood-model/

Part IV

DEEP LEARNING

Part V

SUPERVISED LEARNING

Chapter 13

Introduction

Note: Even though deep learning can be a powerful tool when it comes to fitting a function to data. Unless you have a causal diagram/model, its knowledge is still not tranferable. For example, when you get a new population, you will have to train your deep learning model again.

(LeCun et al., 2015) for a review. (LeCun et al., 2015) defines a deep-learning architecture as "a multilayer stack of simple modules, all (or most) of which are subject to learning, and many of which compute non-linear input-output mappings".

Chapter 14

Neural Network

Neuron

$$z = a_1 w_1 + .. + a_k w_k + b$$

Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Different connections lead to different network structures

The neurons have different values of weights and biases.
Fully connected Feedforward Network

Framework

Step 1: A set of function
Step 2: Goodness of function f
Step 3: Pick the best function

14.1 Step 1: A set of function

Deep = many hidden layers

Universality Theorem
Any continuous, function f

$$f : R^N \rightarrow R^M$$

can be realized by a network with one hidden layer (given enough hidden neurons).

Output layer

- Softmax layer as the output layer

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

  - where
    - σ = softmax
    - $\vec{z}$ = input vector
    - e^{z_i} = standard exponential function for input vector
    - K = number of classes in the multi-class classifier
    - e^{z_j} = standard exponential function for output vector
  - probability:
    - $1 > y_i > 0$
    - $\sum_i y_i = 1$
  - Monotonicity
  - Non-locality

14.2 Step 2: Goodness of function

Loss can be square error or cross entropy between the network output and target.

With softmax, the summation of all the outputs would be one.

A good function should make the loos of all examples as small as possible

Total Loss:

$$L = \sum_{r=1}^{R} l_r$$

Find a function in function set (the network parameters θ^*) that minimizes total loss.

14.3 Step 3: Pick the best function

Find network parameters θ^* that minimize total loss L.

Gradient Descent

Network parameters $\theta = (w_1, w_2, .., b_1, b_2, ...)$

- Pick an initial value for w
- Compute $\frac{\partial L}{\partial w}$,
  - if negative, increase w
  - if positive, decrease w
- $w_{new} = w_{old} - \eta \frac{partialL}{\partial w}$ where η is the learning rate.
- You have problems with:
  - plateau: $\frac{\partial L}{\partial w} \approx 0$
  - saddle point: $\frac{\partial L}{\partial w} = 0$
  - Stuck at local minima: $\frac{\partial L}{\partial w} = 0$
- solution:
  - Different initial point to reach different minima, so different results.

Backpropagation

- is an efficient way to compute $\partial L / \partial w$

14.4 Recipe of Deep learning

1. Not good result on training data

- Choosing proper loss
- Mini-batch
- New activation function
- Adaptive Learning Rate
- Momentum

2. Not good result on testing data

- Early Stopping
- Regularization
- Dropout

- Network Structure

14.4.1 Choosing proper loss

When using softmax output layer, choose cross entropy. Problem

14.4.2 Mini-batch

```
model.fit(x_train,y_train,batch_size = 100,nb_epoch = 20) # 100 examples in a mini-bat
# repeat 20 times
```

- Randomly initialize network parameters
- One epoch
  - Pick the 1st batch $L' = l^1 + l^{31} + ..$ Update parameters once.
  - Pick the 2nd batch $L'' = l^2 + l^{16} + ...$ Update parameters once
  - Until all mini-batches have been picked
- Repeat the above process
- Mini-batch is faster (with mini-batch. if there are 20 batches, update 20 times in one epoch).

14.4.3 New activation function

- Rectified Linear Unit (ReLU)

Reason:

1. Fast to compute
2. Biological reason
3. Infinite sigmoid with different biases
4. Vanishing gradient problem.

Parametric ReLU (PReLU) (Glorot et al., 2011)

ReLU - Variant

- Leaky ReLU
- Parametric ReLu

14.4.4 Adaptive Learning Rate

- Set the learning rate η carefully

- If learning rate is too large
- Total loss may not decrease after each update.
- If learning rate is too small
- Training would be too slow
- Popular and simple idea: reduce the learning rate by some factor every few epochs.
  - At the beginning, we are far from the destination, so we use larger learning rate
  - After several epochs, we are close to the destination, so we reduce the learning rate
  - E.g., 1/t decay $\eta^t = \eta\sqrt{t+1}$
- Learning rate cannot be one-size-fits-all
  - Giving different parameters different learning rates.
- η is unit-dependent (from 0 to 1)

Adagrad

- Original: $w \leftarrow w - \eta \partial L/\partial w$
- Adagrad: $w \leftarrow -\eta_w \partial L/\partial w$
  - where η_w is parameter dependent learning rate.

$$\eta_w = \frac{\eta}{\sqrt{\sum_{i=1}^{t}(g^i)^2}}$$

  - η is constant
  - g^i is $\partial L/\partial w$ obtained at the i-th update.
- Learning rate is smaller and smaller for all parameters
- Smaller derivatives, larger learning rate, and vice versa.

link

14.4.5 Momentum

Hard to find optimal network parameters

Movement = negative of $\partial L/\partial w$ + Momentum

```
# RMSProp (Advanced Adagrad) + Momentum
model.compile(loss='categorial_crossentropy',optimizer = SGD(lr=0.1),metrics = ['accura
model.compile(loss='categorial_crossentropy',optimizer=Adam(),metrics = ['accuracy'])
```

14.4.6 Early Stopping

- training data and testing data can be different.
- Learning target is defined by the training data
- The parameters achieving the learning target do not necessary have good results on the testing data.

14.4.7 Regularization

14.4.8 Dropout

Training

- each time before updating the parameters
  - each neuron has p% to dropout.
    - The structure of the network is changed.
  - Using the new network for training
    - For each min-batch, we resample the dropout neurons.

Testing

- No dropout
  - If the dropout rate training is p%, all the weights times 1-p%
  - Assume that the dropout rate is 50%. If a weight w = 1 by training, set w = 0.5 for testing.

Dropout is a kind of ensemble.

- training of dropout: M neurons $\rightarrow 2^M$ possible networks.
- Using one mini-batch to train one network

- Some parameters int eh network are shared.
- Training of dropout: assume dropout rate is 50%
- Testing of dropout: weight

```
model = Sequential()
model.add(Dense(input_dim = 28*28,output_dim = 500))
model.add(Activation('sigmoid'))
model.add(dropout(0.8))
model.add(Dense (output_dim=500))
model.add(Activation('sigmoid'))
model.add(dropout(0.8))
model.add(Dense(output_dim = 10))
model.add(Activation('softmax'))
```

14.4.9 Network Structure

Convolutional Neural Network (CNN)

- Widely used in image processing
- use in the first step for deep learning

The Whole CNN

- Property 1: Some patterns are much smaller than the whole image
- Property 2: The same patterns appear in different regions.
- Property 3: Subsampling the pixels will not change the object.

CNN in Keras: only modified the network structure and input format (vector to 3-D tensor)

```
model2.add(Convolution2D(25,3,3),input_shape= (1,28,28))
# 25 3x3 filters; 1:black/weight, 3: RFB, 28x28 pixels
model2.add(MaxPooling2D(2,2))
model2.add(Convolution2D(50,3,3))
model2.add(MaxPooling2D(2,2))
model2.add(Flatten())
model2.add(Dense(output_dim = 100))
model2.add(Activation('relu'))
model2.add(Dense(output_dim=10))
model2.add(Activation('softmax'))
```

will we always have to know our classes? Yes, and you have to retrain your model all the time. Or you could use
Transfer learning in deep learning

Neural topic modeling

graph representation learning

GNN

DGL

recurrent network

BERT model

Chapter 15

Overview

- What is deep learning? Input to output via layers of representation
- What are layers? A layer is a geometric transformation function on the data that goes through it Weights determine the data transformation behavior of a layer

Learning Representation

- Transforming input data into useful representation

The “deep” in deep learning

- multiple layers
- Other possibly more appropriate names for the field:
  - Layered representations learning
  - Hierarchical representations learning
  - Chained geometric transformation learning

New problem domains for R:

- Computer vision
- Computer speech recognition
- Reinforcement learning applications

How do we train deep learning models?

- Basic of machine learning algorithms
- Machine learning vs. statistical modeling

- MNIST example
  - Model definition in R
  - Layers of representation
- The training loop

Machine learning algorithms
Learning model parameters via exposure to many example data points

Statistics: Often focused on inferring the process by which data is generated
Machine Learning: Principally focused on predicting future data

Deep learning frontiers

- Computer vision
- Natural language processing
- Time series
- Biomedical

Chapter 16

Tensor Flow and R

TensorFlow APIs

- Keras API
- Estimator API
- Core API

16.1 R packages

TensorFlow APIs * keras: Interface for neural networks, with a focus on enabling fast experimentation. * tfestimators: Implementations of common model types such as regressors and classifiers.
* tensorflow: Low *level interface to the TensorFlow computational graph.* tfdatasets: Scalable input pipelines for TensorFlow models.

Supporting Tools * tfruns : Track , visualize, and manage TensorFlow training runs and experiments * tfdeploy : Tools designed to make exporting and serving TensorFlow models straightforward. * cloudml : R interface to Google Cloud Machine Learning Engine.

R interface to Keras

- Step by step example
- Keras layers
- Compiling models
- Losses, Optimizers, and Metrics
- More examples

16.2 Keras layers

65 layers available

- Dense layers: classic “fully connected” neural network layers
- Convolutional layers: ”Filters for learning local patterns in data
- Recurrent layers: Layers that maintain state based on on previously seen data
- Embedding layers: Vectorization of text that reflects semantic relationships between words

Chapter 17

Compiling models

Model compilation prepares the model for training by:
1. Converting the layers into a TensorFlow graph
2. Applying the specified loss function and optimizer
3. Arranging for the collection of metrics during training

Chapter 18

Losses, Optimizers, and Metrics

All available at Keras for R cheatsheet

Example of Image classificaiton

Chapter 19

NYU

Materials in this chapter is based on NYU Deep Learning

Deep in Deep Learning means multi-layers. "A deep network has several layers and uses them to build a hierarchy of features of increasing complexity"

Supervised Learning (= Function Optimization): With inputs and outputs, you train the machine to tweak its parameter to have the correct outputs from inputs. In "traditional machine learning", you have to engineer **feature extractor**, but in deep learning you can multi-layers feature extractor can be trained. And all layers are non-linear, because if they are linear, then all layers would be collapse into one layer.

Natural Data is compositional. Hence, it is efficiently representable hierarchically.

Revolutions in Deep Learning:

- Speech Recognition: 2010
- Image Recognition: 2013
- Natural Language Processing (NLP): 2015

Deep Learning = Learning Representations/ Features Representation means you turn raw data into something useful

Image Recognition:
Pixel, edge, texton, motif, part, object

Text
Character, group, word group, clause, sentence, story

Speech
Sample, spectral band, sound, phone, phoneme, word.

Difference between Deep Learning Support-Vector Machines (SVM)

SVM is a 2-layer neural net, where

- layer 1 = templates
- layer 2 = linear classifier.

where Deep Learning is "A deep network has several layers and uses them to build a hierarchy of features of increasing complexity"

Deep Machines are more efficient than SVM in representing certain classes of functions.

The Manifold Hypothesis "states that real-world high-dimensional data lie on low-dimensional manifolds embedded within the high-dimensional space"

Difference between Deep Learning and PCA (Principal Components Analysis) is that Deep learning uses non-linear function, while PCA uses linear function.

Chapter 20

Applications

```
# Step 1: define a set of function
model = Sequential()
model.add(Dense(input_dim = 28*28,
output_dim = 500))

model.add(Activation('sigmoid'))
model.add(Dense(output_dim = 500))
model.add(Activation('sigmoid'))
model.add(Dense(output_dim=10))
model.add(Activation('softmax'))

# Step 2: goodness of function
model.compile(loss = 'mse',optimizer = SGD(lr=0.1),metrics = ['accuracy'])

# Step 3: Pick the best function
# Step 3.1: Configuration
model.compile(loss = 'mse',optimizer=SGD(lr=0.1),metrics = ['accuracy'])

# Step 3.2: Find the optimal network parameters
model.fit(x_train,y_train,batch_size = 100,nb_epoch=20) # x_train = training data (images)
# y_train: labels (digits)


# Save and load models
# http://keras.io/getting-started/faq/#how-can-i-save-a-keras-model

# How to use the neural network (testing):
# case 1
score = model.evaluate(x_test,y_test)
```

```
print('Total loss on Testing Set:',score[0])
print('Accuracy of Testing set:',score[1])

# case 2
result = model.predict(x_test)
```

Chapter 21

Research Methods

21.1 But-for World

(Mahajan et al., 1993) (Landsman and Stremersch, 2020)

Chapter 22

Model Card

Originate from the (Mitchell et al., 2019) Google's Model Card. This is scientists' attempt to bring transparency, fairness and equity to other stakeholders that are affected by AI models. Two specific examples that serve as a template are:

- Face Detection
- Object Detection

Although we have pre-defined template in Python and example, and deploy, there doesn't seem to be one in R

Chapter 23

Model Tuning

```
library("finetune")
```

```
## Warning: package 'finetune' was built under R version 4.0.5
```

```
## Loading required package: tune
```

```
## Warning: package 'tune' was built under R version 4.0.5
```

```
## Registered S3 method overwritten by 'tune':
##   method                   from
##   required_pkgs.model_spec parsnip
```

(Kuhn, 2014) for detail

- Grid Search
- Annealing Search

Chapter 24

Model Stacking

Codes are from stack vignettes basic and

Source Code

Steps in doing model staking:

1. Define your candidate (ensemble) models
2. Initialize a `data_stack` object with `stacks()`
3. Add candidate candidate models using `add_candidates()`
4. Evaluate the combined predictions `blend_predictions()`
5. 'fit_members()' allows you to fit models with non-zero staking coefficients
6. `predict()` new data

```
library(tidymodels)
```

```
## Warning: package 'tidymodels' was built under R version 4.0.5
```

```
## -- Attaching packages ------------------------------------- tidymodels 0.1.3 --
```

```
## v broom        0.7.9      v recipes      0.1.16
## v dials        0.0.10     v rsample      0.1.0
## v infer        1.0.0      v workflows    0.2.3
## v modeldata    0.1.1      v workflowsets 0.1.0
## v parsnip      0.1.7      v yardstick    0.0.8
```

```
## Warning: package 'broom' was built under R version 4.0.5
```

```
## Warning: package 'dials' was built under R version 4.0.5
```

```
## Warning: package 'modeldata' was built under R version 4.0.5
```

```
## Warning: package 'parsnip' was built under R version 4.0.5
```

```
## Warning: package 'recipes' was built under R version 4.0.5
```

```
## Warning: package 'rsample' was built under R version 4.0.5
```

```
## Warning: package 'workflows' was built under R version 4.0.5
```

```
## Warning: package 'workflowsets' was built under R version 4.0.5
```

```
## Warning: package 'yardstick' was built under R version 4.0.5
```

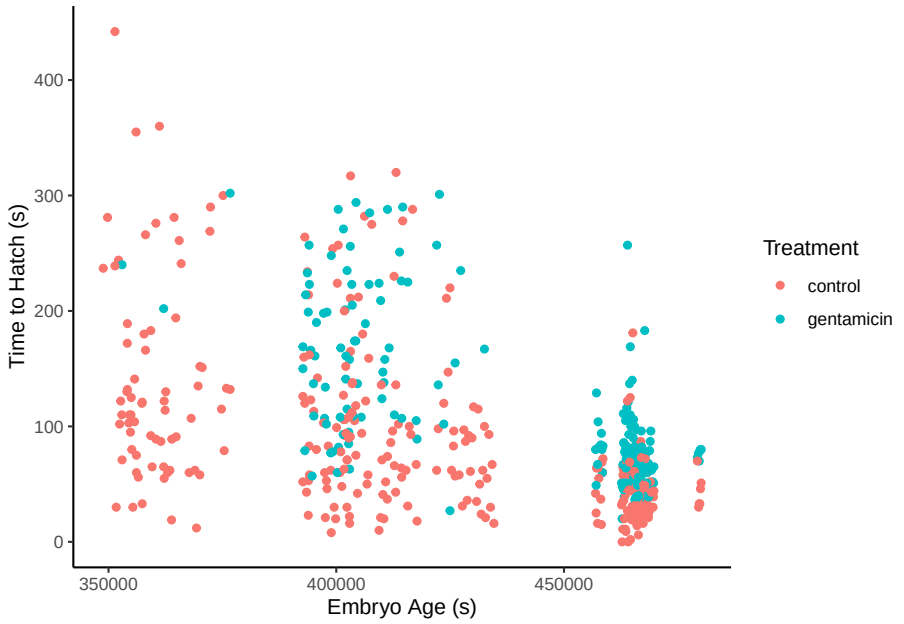
```
## -- Conflicts ---------------------------------------- tidymodels_conflicts() --
## x nlme::collapse()         masks dplyr::collapse()
## x scales::discard()        masks purrr::discard()
## x dplyr::filter()          masks stats::filter()
## x recipes::fixed()         masks stringr::fixed()
## x dplyr::lag()             masks stats::lag()
## x caret::lift()            masks purrr::lift()
## x yardstick::precision()   masks caret::precision()
## x yardstick::recall()      masks caret::recall()
## x yardstick::sensitivity() masks caret::sensitivity()
## x yardstick::spec()        masks readr::spec()
## x yardstick::specificity() masks caret::specificity()
## x recipes::step()          masks stats::step()
## * Use tidymodels_prefer() to resolve common conflicts.
```

```
library(stacks)
```

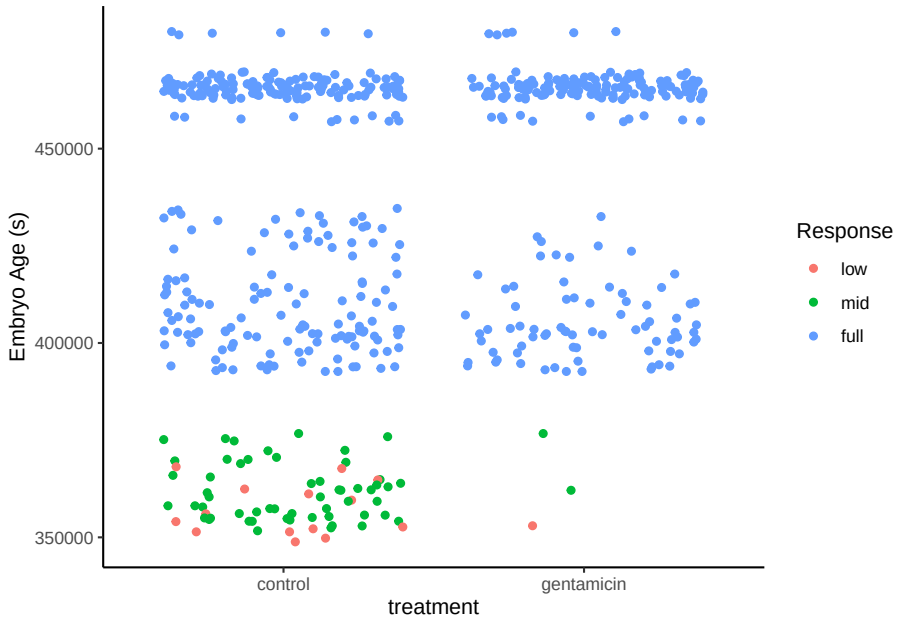
```
## Warning: package 'stacks' was built under R version 4.0.5
```

```
library(dplyr)
library(purrr)
library(tune)
```

```
data("tree_frogs")
# subset the data
tree_frogs <- tree_frogs %>%
    filter(!is.na(latency)) %>%
    select(-c(clutch, hatched))
library(ggplot2)

ggplot(tree_frogs) +
    aes(x = age, y = latency, color = treatment) +
    geom_point() +
    labs(x = "Embryo Age (s)", y = "Time to Hatch (s)", col = "Treatment")
```




```
ggplot(tree_frogs) +
    aes(x = treatment, y = age, color = reflex) +
    geom_jitter() +
    labs(y = "Embryo Age (s)",
         x = "treatment",
         color = "Response")
```




```
# some setup: resampling and a basic recipe
set.seed(1)

tree_frogs_split <- initial_split(tree_frogs)
tree_frogs_train <- training(tree_frogs_split)
tree_frogs_test  <- testing(tree_frogs_split)

folds <- rsample::vfold_cv(tree_frogs_train, v = 5)

tree_frogs_rec <-
    recipe(reflex ~ ., data = tree_frogs_train) %>%
    step_dummy(all_nominal(),-reflex) %>%
    step_zv(all_predictors())

tree_frogs_wflow <-
    workflow() %>%
    add_recipe(tree_frogs_rec)
ctrl_grid <- control_stack_grid() # same control settings
ctrl_res <- control_stack_resamples()
```

Add random forest model

```
rand_forest_spec <-
    rand_forest(mtry = tune(),
                min_n = tune(),
                trees = 500) %>%
```

```
    set_mode("classification") %>%
    set_engine("ranger")

rand_forest_wflow <-
    tree_frogs_wflow %>%
    add_model(rand_forest_spec)

rand_forest_res <-
    tune_grid(
        object = rand_forest_wflow,
        resamples = folds,
        grid = 10,
        control = ctrl_grid
    )
```

```
## i Creating pre-processing data to finalize unknown parameter: mtry
```

```
## Warning: package 'rlang' was built under R version 4.0.5
```

```
## Warning: package 'vctrs' was built under R version 4.0.5
```

```
## Warning: package 'ranger' was built under R version 4.0.5
```

```
## ! Fold2: internal: No observations were detected in `truth` for level(s): 'low'
## C...
```

```
## ! Fold4: internal: No observations were detected in `truth` for level(s): 'low'
## C...
```

Add neural network model

```
nnet_spec <-
    mlp(hidden_units = tune(),
        penalty = tune(),
        epochs = tune()) %>%
    set_mode("classification") %>%
    set_engine("nnet")

nnet_rec <-
    tree_frogs_rec %>%
    step_normalize(all_predictors())

nnet_wflow <-
    tree_frogs_wflow %>%
    add_model(nnet_spec)

nnet_res <-
    tune_grid(
        object = nnet_wflow,
```

```
        resamples = folds,
        grid = 10,
        control = ctrl_grid
    )
```

```
## Warning: package 'nnet' was built under R version 4.0.5
```

```
## ! Fold2: internal: No observations were detected in `truth` for level(s): 'low'
## C...
```

```
## ! Fold4: internal: No observations were detected in `truth` for level(s): 'low'
## C...
```

Build stacks

```
tree_frogs_model_st <-
    # initialize the stack
    stacks() %>%
    # add candidate members
    add_candidates(rand_forest_res) %>%
    add_candidates(nnet_res) %>%
    # determine how to combine their predictions
    blend_predictions() %>%
    # fit the candidates with nonzero stacking coefficients
    fit_members()
```

```
## Warning: package 'glmnet' was built under R version 4.0.5
```

```
## Warning: package 'Matrix' was built under R version 4.0.5
```

```
## ! Bootstrap01: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -52
```

```
## ! Bootstrap02: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -59
```

```
## ! Bootstrap03: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -61
```

```
## ! Bootstrap04: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -54
```

```
## ! Bootstrap05: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -52
```

```
## ! Bootstrap06: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -57
```

```
## ! Bootstrap07: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -56
```

```
## ! Bootstrap08: internal: No observations were detected in `truth` for level(s): 'lou
```

```
## ! Bootstrap09: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -55
```

```
## ! Bootstrap10: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -59
```

```
## ! Bootstrap11: preprocessor 1/1, model 1/1: one multinomial or binomial class has fe
```

```
## ! Bootstrap12: preprocessor 1/1, model 1/1: one multinomial or binomial class has fe
```

```
## ! Bootstrap13: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -58
```

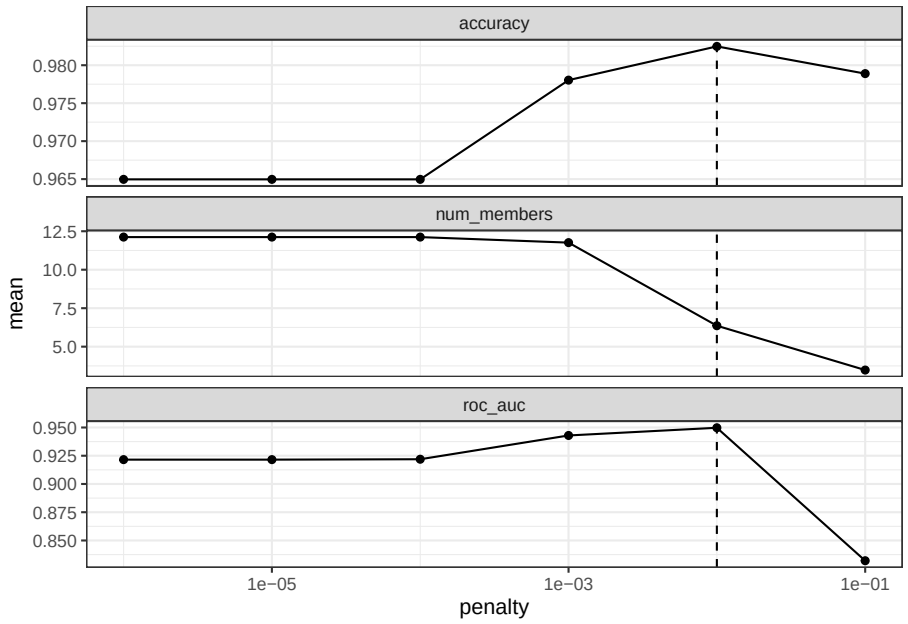
```
## ! Bootstrap14: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -59); ...
## ! Bootstrap15: preprocessor 1/1, model 1/1: one multinomial or binomial class has fewer...
## ! Bootstrap16: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -60); ...
## ! Bootstrap17: internal: No observations were detected in `truth` for level(s): 'low', ...
## ! Bootstrap18: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -53); ...
## ! Bootstrap19: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -59); ...
## ! Bootstrap20: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -59); ...
## ! Bootstrap21: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -58); ...
## ! Bootstrap22: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -50); ...
## ! Bootstrap23: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -51); ...
## ! Bootstrap24: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -52); ...
## ! Bootstrap25: preprocessor 1/1, model 1/1: from glmnet Fortran code (error code -60); ...
## Warning: from glmnet Fortran code (error code -55); Convergence for 55th lambda
## value not reached after maxit=100000 iterations; solutions for larger lambdas
## returned
```

```
tree_frogs_model_st
```

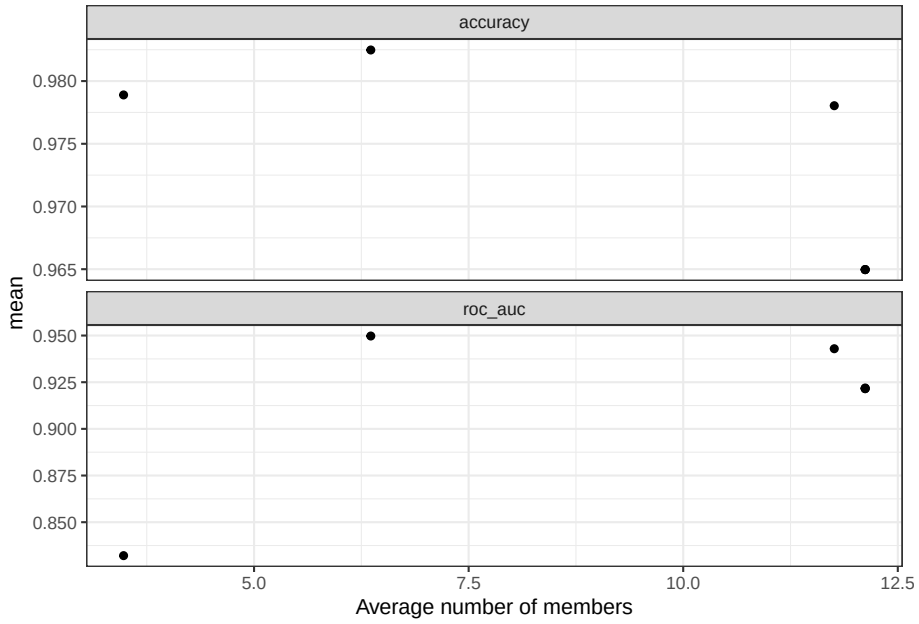
```
## -- A stacked ensemble model -------------------------------------
##
## Out of 40 possible candidate members, the ensemble retained 5.
## Penalty: 0.01.
## Mixture: 1.
## Across the 3 classes, there are an average of 1.67 coefficients per class.
##
## The 5 highest weighted member classes are:
## # A tibble: 5 x 4
##   member                           type        weight class
##   <chr>                            <chr>        <dbl> <chr>
## 1 .pred_mid_nnet_res_1_10          mlp         12.7   low
## 2 .pred_mid_nnet_res_1_09          mlp          7.25  mid
## 3 .pred_full_rand_forest_res_1_10 rand_forest  6.62  full
## 4 .pred_mid_rand_forest_res_1_07  rand_forest  1.89  mid
## 5 .pred_full_rand_forest_res_1_07 rand_forest  0.586 full
```

Check performance

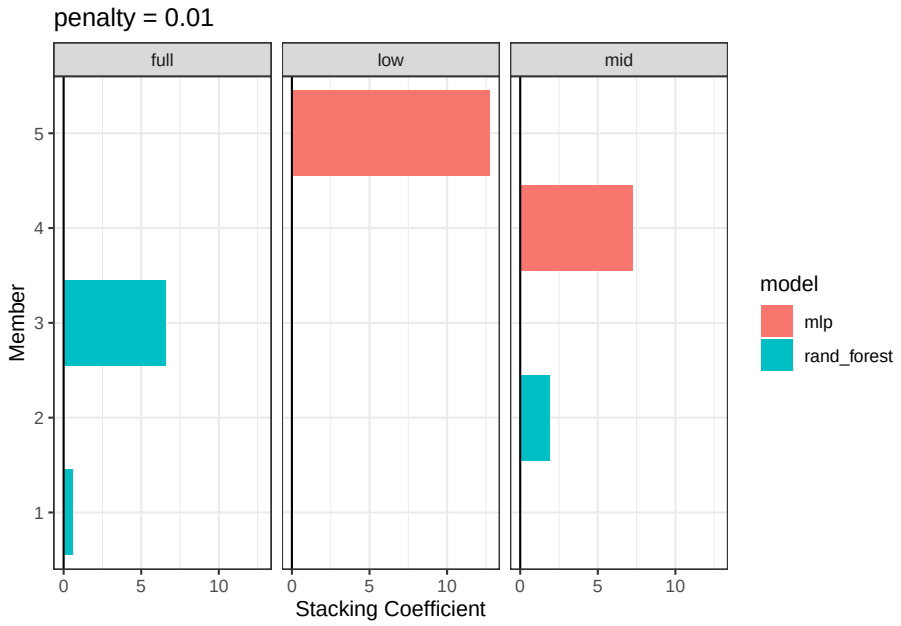
```
theme_set(theme_bw())
autoplot(tree_frogs_model_st)
```




```
autoplot(tree_frogs_model_st, type = "members")
```




```
autoplot(tree_frogs_model_st, type = "weights") # to see penalty
```



See parameters

```
collect_parameters(tree_frogs_model_st, "rand_forest_res")
```

```
## # A tibble: 60 x 6
##    member                mtry min_n class terms                          coef
##    <chr>                <int> <int> <chr> <chr>                         <dbl>
##  1 rand_forest_res_1_01     1    32 low   .pred_mid_rand_forest_res_1_01      0
##  2 rand_forest_res_1_01     1    32 low   .pred_full_rand_forest_res_1_01     0
##  3 rand_forest_res_1_01     1    32 mid   .pred_mid_rand_forest_res_1_01      0
##  4 rand_forest_res_1_01     1    32 mid   .pred_full_rand_forest_res_1_01     0
##  5 rand_forest_res_1_01     1    32 full  .pred_mid_rand_forest_res_1_01      0
##  6 rand_forest_res_1_01     1    32 full  .pred_full_rand_forest_res_1_01     0
##  7 rand_forest_res_1_02     1    27 low   .pred_mid_rand_forest_res_1_02      0
##  8 rand_forest_res_1_02     1    27 low   .pred_full_rand_forest_res_1_02     0
##  9 rand_forest_res_1_02     1    27 mid   .pred_mid_rand_forest_res_1_02      0
## 10 rand_forest_res_1_02     1    27 mid   .pred_full_rand_forest_res_1_02     0
## # ... with 50 more rows
```

Predict new data

```
tree_frogs_pred <-
    tree_frogs_test %>%
    bind_cols(predict(tree_frogs_model_st, .,type = "prob"))
```

ROC AUC

```
yardstick::roc_auc(tree_frogs_pred,
                   truth = reflex,
                   contains(".pred_"))
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc hand_till      0.364
```

Compare predictions with actual observations

```
# ggplot(tree_frogs_pred) +
#     aes(x = latency,
#         y = .pred) +
#     geom_point() +
#     coord_obs_pred()
```

Comparison between stacks and member model

```
member_preds <-
    tree_frogs_test %>%
    select(latency) %>%
    bind_cols(predict(tree_frogs_model_st, tree_frogs_test, members = TRUE))
```

Compare different model predictions

```
map_dfr(setNames(colnames(tree_frogs_pred), colnames(tree_frogs_pred)),
        ~ mean(tree_frogs_pred$reflex == pull(tree_frogs_pred, .x))) %>%
    pivot_longer(c(everything(),-reflex))
```

```
## # A tibble: 7 x 3
##   reflex name       value
##    <dbl> <chr>      <dbl>
## 1      1 treatment      0
## 2      1 age            0
## 3      1 t_o_d          0
## 4      1 latency        0
## 5      1 .pred_full     0
## 6      1 .pred_low      0
## 7      1 .pred_mid      0
```

Chapter 25

Data Storage

	Structured Databases	Unstructured Databases
Structure	every element has the same number of attributes (i.e., column)	Different elements can have different number of attributes (more efficient)
Latency	Slower	Faster
Ease of learning	Easy	Harder, more steep
Storage Volume	Not appropriate for strong Big Data	handle Big Data well
Data Types	numerical, texture	any (e.g., audio, video)
Examples	MySQL, PostgreSQL	MongoDB, Neo4j

25.1 Structured Databases

25.2 Unstructured Databases

25.2.1 MongoDB

```
import pymongo

#%% Create a Database
myclient = pymongo.MongoClient("mongodb://localhost:27017/")

mydb = myclient["mydatabase"]
```

```
#%% Check if database exists

#check if a database exist by listing all databases in you system
print(myclient.list_database_names())

#check a specific database by name
dblist = myclient.list_database_names()
if "mydatabase" in dblist:
  print("The database exists.")

#%% Insert Into Collection
#The first parameter of the insert_one() method is a dictionary containing the name(s)


myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

mydict = { "name": "John", "address": "Highway 37" }

x = mycol.insert_one(mydict)

#%% Return the _id field
mydict = { "name": "Peter", "address": "Lowstreet 27" }

x = mycol.insert_one(mydict)

print(x.inserted_id)

#%% Insert Multiple documents
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

mylist = [
  { "name": "Amy", "address": "Apple st 652"},
  { "name": "Hannah", "address": "Mountain 21"},
  { "name": "Michael", "address": "Valley 345"},
  { "name": "Sandy", "address": "Ocean blvd 2"},
  { "name": "Betty", "address": "Green Grass 1"},
  { "name": "Richard", "address": "Sky st 331"},
  { "name": "Susan", "address": "One way 98"},
  { "name": "Vicky", "address": "Yellow Garden 2"},
```

```
  { "name": "Ben", "address": "Park Lane 38"},
  { "name": "William", "address": "Central st 954"},
  { "name": "Chuck", "address": "Main Road 989"},
  { "name": "Viola", "address": "Sideway 1633"}
]

x = mycol.insert_many(mylist)

#print list of the _id values of the inserted documents:
print(x.inserted_ids)

#%% Insert Multiple Documents, with Specified IDs
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

mylist = [
  { "_id": 1, "name": "John", "address": "Highway 37"},
  { "_id": 2, "name": "Peter", "address": "Lowstreet 27"},
  { "_id": 3, "name": "Amy", "address": "Apple st 652"},
  { "_id": 4, "name": "Hannah", "address": "Mountain 21"},
  { "_id": 5, "name": "Michael", "address": "Valley 345"},
  { "_id": 6, "name": "Sandy", "address": "Ocean blvd 2"},
  { "_id": 7, "name": "Betty", "address": "Green Grass 1"},
  { "_id": 8, "name": "Richard", "address": "Sky st 331"},
  { "_id": 9, "name": "Susan", "address": "One way 98"},
  { "_id": 10, "name": "Vicky", "address": "Yellow Garden 2"},
  { "_id": 11, "name": "Ben", "address": "Park Lane 38"},
  { "_id": 12, "name": "William", "address": "Central st 954"},
  { "_id": 13, "name": "Chuck", "address": "Main Road 989"},
  { "_id": 14, "name": "Viola", "address": "Sideway 1633"}
]

x = mycol.insert_many(mylist)

#print list of the _id values of the inserted documents:
print(x.inserted_ids)

#%% Find One

import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
```

```
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

x = mycol.find_one()

print(x)

#%% Find All


import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

for x in mycol.find():
  print(x)

#%% Return Only Some Fields

import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

for x in mycol.find({},{ "_id": 0, "name": 1, "address": 1 }):
  print(x)

#%% Example exclude "address"

import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

for x in mycol.find({},{ "address": 0 }):
  print(x)


#%% get error if you specify both
import pymongo
```

```
myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

for x in mycol.find({},{ "name": 1, "address": 0 }):
  print(x)


#%%  Filter the Result

import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

myquery = { "address": "Park Lane 38" }

mydoc = mycol.find(myquery)

for x in mydoc:
  print(x)


#%% Advanced Query

import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

myquery = { "address": { "$gt": "S" } } # o find the documents where the "address" field starts u

mydoc = mycol.find(myquery)

for x in mydoc:
  print(x)

#%% Filter With Regular Expressions
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]
```

```
myquery = { "address": { "$regex": "^S" } } #Find documents where the address starts w

mydoc = mycol.find(myquery)

for x in mydoc:
  print(x)


#%% Sort the Result

import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

mydoc = mycol.find().sort("name") #method takes one parameter for "fieldname" and one p

for x in mydoc:
  print(x)

#%% Sort Descending

import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

mydoc = mycol.find().sort("name", -1) #Sort the result reverse alphabetically by name:

for x in mydoc:
  print(x)

#%% Delete Document

import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

myquery = { "address": "Mountain 21" } #delete document with address "Mountain 21"

mycol.delete_one(myquery)
```

```
#%% Delete Many Documents
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

myquery = { "address": {"$regex": "^S"} } #Delete all documents were the address starts with the

x = mycol.delete_many(myquery)

print(x.deleted_count, " documents deleted.")

#%% Delete All Documents in a Collection
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

x = mycol.delete_many({}) # Delete all documents in the "customers" collection:

print(x.deleted_count, " documents deleted.")

#%% Delete Collection

import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"] #Delete the "customers" collection:

mycol.drop()
# The drop() method returns true if the collection was dropped successfully, and false if the col

#%% Update Collection
# If the query finds more than one record, only the first occurrence is updated.
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

myquery = { "address": "Valley 345" } #Change the address from "Valley 345" to "Canyon 123":
newvalues = { "$set": { "address": "Canyon 123" } }
```

```
mycol.update_one(myquery, newvalues)

#print "customers" after the update:
for x in mycol.find():
  print(x)

#%% Update Many
  #o update all documents that meets the criteria of the query, use the update_many() m

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

myquery = { "address": { "$regex": "^S" } } #Update all documents where the address st
newvalues = { "$set": { "name": "Minnie" } }

x = mycol.update_many(myquery, newvalues)

print(x.modified_count, "documents updated.")

#%% Limit the Result
# Limit the result to only return 5 documents:
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

myresult = mycol.find().limit(5)

#print the result:
for x in myresult:
  print(x)
```

Chapter 26

API

26.1 R

Example

check Blair's talk

https://www.rplumber.io/ https://community.rstudio.com/tag/plumber https://github.com/rstudio/webinars https://github.com/meztez/rapidoc https://github.com/meztez/plumberDeploy https://github.com/rstudio/plumber https://github.com/sol-eng/plumberExamples

Chapter 27

BERT

https://github.com/google-research/bert

https://www.freecodecamp.org/news/google-bert-nlp-machine-learning-tutorial/

https://jalammar.github.io/a-visual-guide-to-using-bert-for-the-first-time/

https://towardsml.com/2019/09/17/bert-explained-a-complete-guide-with-theory-and-tutorial/

Chapter 28

Generalized Method of Moments

Chapter 29

Supervised Learning

From the data, find a function f of the inputs that best approximates the outputs.

$$\hat{Y} = f(X_1, .., X_n)$$

- Predictive: Make predictions for a new object described by its attributes
- Informative: help understand the relationship between inputs and outputs

Divide data into

- Training set: train model
- Validation set: tune hyper parameters
- Test set: evaluation of the model for generalization error.

Learning algorithm

- defined by
  - a family of candidate models (= **hypothesis space H**)
  - a quality measure for a model
  - an optimization strategy
- takes as input a leaning sample and outputs a function h in H of maximum quality

Model selection

- Why not choose the model that minimizes the error rate on the learning sample? (**re-substitution error**)
- How well are you going to predict future data drawn from the same distribution? (**generalization error**)
- Evaluation on test set

1. Randomly choose a percentage (e.g., 30%) of the data to be in a test set
2. The remainder is training set
3. Learn the model from the training set
4. Estimate its future performance on the test set.

- Problems:
  - overfits: occurs when the learning algorithm starts fitting noise.
  - underfits
- Evaluation on test set:
  - Good:
    - Very simple
    - Computationally efficient
  - Bad:
    - Waste data
    - very unstable when database is small
- Another evaluation: **Leave-one-out cross validation**
  - For k = 1 to N
    - remove the k-th object from the learning sample
    - learn the model on the remaining objects
    - apply the model to get a prediction for the k-th object
  - report the proportion of mis-classified objects

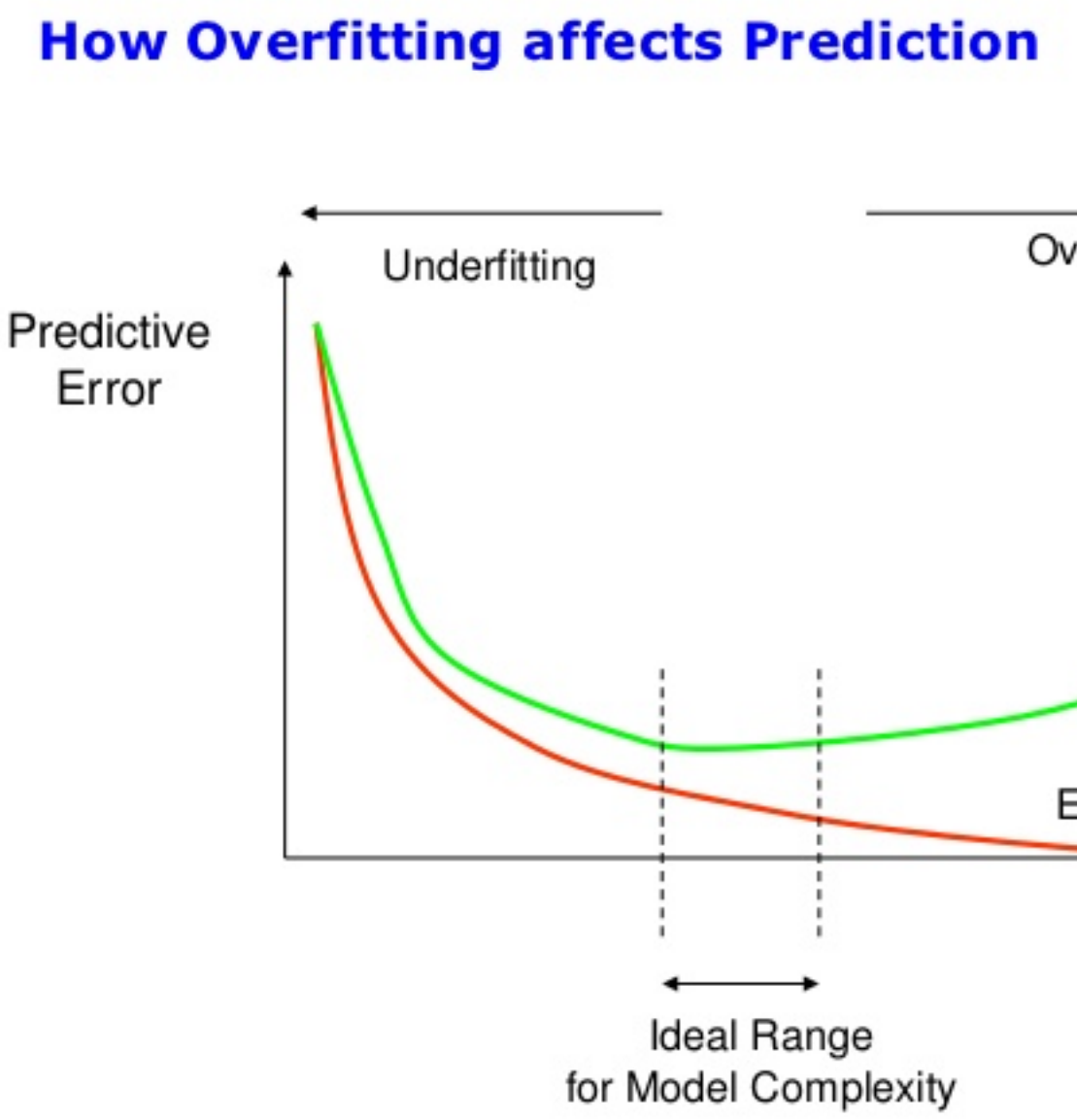
- Good:
  - Do not waste the data (you get an estimate of the best method to apply to N-1 data)
- Bad:
  - Computationally intensive (train N models)
  - high variance.
- Another evaluation: **K-fold cross validation**:
  - Break up data into k folds
  - For each fold
    - Choose the fold as a temporary test set
    - Train on k-1 folds, compute performance on the test fold.
  - Report average performance of the 10 runs.
  - randomly partition the dataset into k subsets (e.g., 10)

- For each subset:
    - learn the model on the objects that are not in the subset
    - compute the error rate on the points in the subset. Report the mean error rate over the k subsets.
- When k = the number of objects -> leave-one-out-cross validation.

Rule-of-thumb for evaluation

- Lots of data (>1000) test set validation
- small data (100-1000): 10-fold CV
- very small data (<100) leave one out CV.

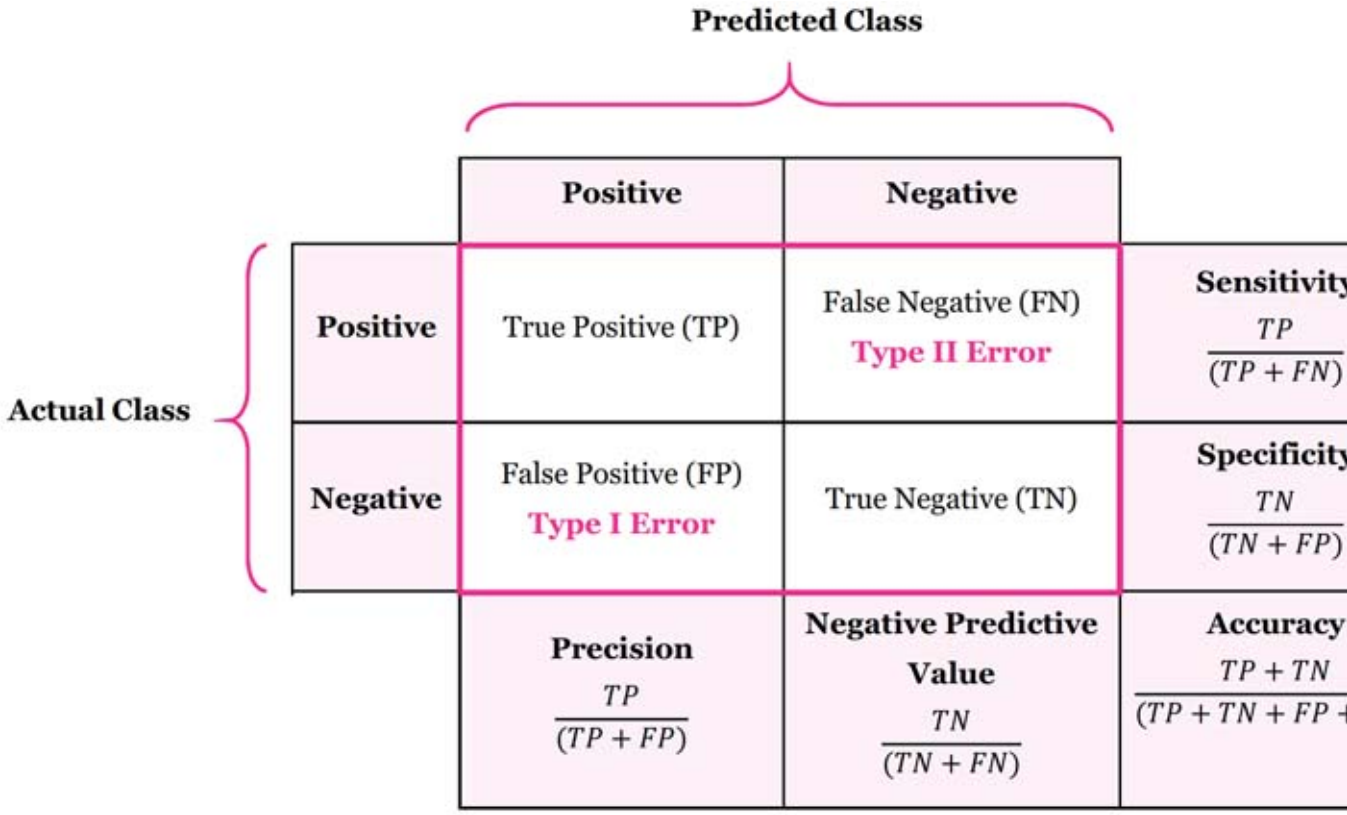
```
## Warning: package 'jpeg' was built under R version 4.0.5
```


How Overfitting affects Prediction
Underfitting
Predictive
Error
Ideal Range
for Model Complexity

Use the testing set to estimate the performance of the final model

Performance measures

- The error rate is no the only way to assess a predictive model.
- In binary classification, results can be summarized in a contingency table (i.e., confusion matrix)



Various criteria

$$\text{Error rate} = \frac{FP+FN}{N+P} \quad \text{Accuracy} = \frac{TP+TN}{N+P} = 1 - \text{Error rate} \quad \text{Sensitivity or recall} = \frac{TP}{P} \quad \text{Specificity} = \frac{TN}{TN+FP} \quad \text{Pr}$$

ROC and precision/recall curves

- Each point corresponds to a particular choice of the decision threshold.

Also known as robustness check

You want large area under the curve

Compassion of learning models

- Three main criteria:
  - Accuracy: Measures by the generalization error (estimated by CV)
  - Efficiency: Computing times and scalabiility for learning and testing
  - Interpretability: Comprehension brought by the model about the input-output relationship.
- Depends on the application, you have to make trade-off.

Learning Techniques

- Supervised learning categories and techniques
  - Linear classifier (numerical functions):
    - Perceptron
    - Logistic regression
    - Support vector machine (SVM): linear to nonlinear: feature transform and kernel function
    - Ada-line
    - Multi-layer perceptron (MLP)
  - Parametric (Probabilistic functions)
    - Naive Bayes, Gaussian discriminant analysis (GDA), Hidden Markov models (HMM), probabilistic graphical model
  - Non-parametric (Instance-based functions)
    - K-nearest neighbors, Kernel regression, Kernel density estimation, local regression.
  - Non-metric (symbolic functions)
    - Classification and regression tree (CART), decision tree
  - Aggregation
    - Bagging (bootstrap + aggregation), Adaboost, Random forest.
- Unsupervised learning categories and techniques:
  - Clustering
    - K-means clustering
    - Spectral clustering
  - Density Estimation
    - Gaussian mixture model (GMM)
    - Graphical models

  - Dimensionality reduction
    * Principal component analysis (PCA)
    * Factor analysis

- Semi-supervised learning
  - goal: exploit both labeled and unlabeled data to build better models than using each one alone
  - Self-training:
    * Iteratively label some unlabeled examples with a model learned from the previously labeled examples
    * Active learning.

- Reinforcement learning
  - Learning from interactions
  - Goal: learn to choose sequence of actions (=policy) that maximizes $r_0 + \gamma r_1 + \gamma^2 r_2 + ...$ where $0 \leq \gamma < 1$

29.1 Decision Tree

- Is a tree-structured plan of a set of attributes to test to predict the output.
- Give one attribute, try to predict the value of new people's attribute by means of some other available attribute
- Input attributes:
  - **d** features/ attributes $x^{(1)}, x^{(2)}, ..., x^{(d)}$
  - Each $x^{(j)}$ has domain O_j
    * Categorical $O_j = (read, blue)$
    * Numerical: $H_j = (0, 10)$
  - Y is output variable with domain O_Y:
    * Categorical: Classification
    * Numerical: regression
- Data D:
  - n examples (x_i, y_i) where
    * x_i is a d-dim feature vector
    * $y_i \in O_Y$
- Decision trees:
  - Split the data at each internal node
  - Each leaf node makes a prediction

How to construct a tree

- Find best split
- Stopping Criteria
- Find Prediction

29.1.1 Find best split

Pick an attribute and value that optimizes some criterion
+ Regression: Purity + Classification: Information Gain: Measures how much a given attribute X tells us about the class Y.

Information Gain? Entropy

The entropy of X: $H(X) = -\sum_{j=1}^{m} p_j log p_j$

- "High entropy": X is from a uniform distribution
  - A histogram of the frequency distribution of values of X is flat
- "Low Entropy": X is from varied distribution
  - A histogram of the frequency distribution of values of X would have many lows and one or two highs.
- Def: Information Gain:
  - $IG(Y|X) = H(Y) - H(Y|X)$ I must transmit Y. How many bits on average would it save me if both ends of the line knew X?

29.1.2 Stopping Criteria

- When the leaf is "pure": the target variable does not vary too much: Var(Y) $<$ threshold.
- When number of examples in the leaf is too small.
- Could also use information as as a stopping criteria.

29.1.3 Find Prediction

- Regression: Predict average y of the examples
- Classification: Predict most common y of the examples the leaf.

29.2 Random Forest

Ensemble methods

- A single decision tree does not perform well.
- But, it is super fast.
- What if we learn multiple trees?

Bagging

- If we split the data in random different ways, decision trees give different results, high variance.
- Bagging: Bootstrap aggregating is a method that result in low variance.
- If we had multiple realizations of the data (or multiple samples) we could calculate the prediction multiple times and take the average of the fact that averaging multiple onerous estimations produce less uncertain results
- say for each sample b, we calculate $f^b(x)$, then $\hat{f}_{avg}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^b(x)$
- Bootstrap: Construct B (hundreds) of trees. Learn a classifier for each bootstrap sample and average them. Very effective.
- Reduces overfitting (variance)
- Could use one classifier, or multiple classifiers.
- Easy to parallelize
- Variable Importance Measures
  - Bagging results in improved accuracy over prediction using a single tree
  - Unfortunately, difficult to interpret the resulting model. Bagging improves prediction accuracy at the expense of interpretability.
- Issues:
  - Each tree is identically distributed (i.d)
    - The expectation of the average of B such trees
    - The bias of bagged trees is the same as that of individual trees
  - An average of B i.i.d random variables, each with variance σ^2, has variance σ^2/B. If i.d. (identical but not independent) and pair correlation ρ is present, then the variance is : $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. As B increases, the second term disappears but the first term remains.

- Suppose that there is one very strong predictor in the data set, along with a number of other moderately strong predictors. Then all bagged trees will select the strong predictor at the top tree and therefore all trees will look similar.
  - We can penalize the splitting with a penalty term that depends on the number of times a predictor is selected at a given length.
  - restrict how many items a predictor can be used.
  - allow a certain number of predictors.
- Since we want i.i.d so that the bias to be the same and variance to be less.
  - in bagging, we build a number of decision trees on bootstrapped training samples each time a split in a tree is considered, a random sample of m predictors is chosen as split candidates from the full set of p predictors (if m = p, then this is bagging).

Random Forest software

Random Forests Algorithm

For b = 1 to B:

a. draw a bootstrap sample Z * of size N from the training data.

b. Grow a random-forest tree to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.
   - Select m variables at random from the p variables.
   - Pick the best variable/split-point among the m.
   - Split the node into two daughter nodes.

Output the ensemble trees.

	Regression	Classification
To make a prediction at a new point x	average the results	majority vote
Tuning	The default value for m is $p/3$ and the minimum node size is 5	The default value for m is $\sqrt{p}$ and the minimum node size is one

Another issue:

When the number of variables is large, but the fraction of relevant variables is small, random forests are likely to perform poorly when m is small. Because: At each split the chance can be small that the relevant variables will be selected.

Example: with 3 relevant and 100 not so relevant variables the probability of any of the relevant variables being selected at any split is approx 0.25.

random forests "cannot overfit" the data because the number of trees (B) does not mean increase in the flexibility of the model.

Always include Random Forest (ensembles) and XGBoost (like bagging)

Alternative

- Support vector machine (SVM) dont usually use this because of computational reasons and because it is only 1 model.

Chapter 30

Unsupervised Learning

Unsupervised learning tries to find regularities in the data without guidance about inputs and outputs

Unsupervised learning categories and techniques:

- Clustering
  - K-means Clustering
  - Spectral Clustering
- Density Estimation
  - Gaussian mixture model (GMM)
  - Graphical models
- Dimensionality reduction
  - Principal component analysis (PCA)
  - Factor Analysis

30.1 Clustering

Clustering is used to find similar groups in data (i.e., clusters)

No class value denoting an a priori grouping (hence, Unsupervised Learning)

Clustering algorithm

- Partitional clustering
- Hierarchical clustering

A distance (similarity, or dissimilarity) function

Clustering quality

- Inter-clusters distance -> maximized
- Intra-clusters distance -> minimized

which can be use to judge good clustering

- Internal criterion: A good clustering will produce high quality clusters in which:
  - The intra-class (intra-cluster) similarity is high
  - The inter-class similarity is low.
  - The measured quality of a clustering depends on both the document representation and the similarity measure used.
- External criteria:
  - Quality measured by its ability to discover some or all of the hidden patterns or latent classes in gold standard data.
  - Assesses a clustering with respect to **ground truth**
  - Assume documents with C gold standard classes, while our clustering algorithm produce cluster $w_1, w_2, ..., w_k$ with $n_1, ..., n_k$ members.
  - Simple measure: **purity**, the ratio between the dominant class in the cluster π_i and the size of cluster w_i

$$Purity(w_i) = \frac{1}{n_i} max_j(n_{ij}) j \in C$$

  - others are entropy of classes in clusters (or mutual inforamtion between classes and clusters).

The quality of a clustering result depends on

- The algorithm
- the distance function
- the application

30.1.1 Similarity Measures

It's your choice of how to define similarity

(Dis)similarity measure

- Equivalently, we can use dissimilarity measures
- Jagota defines a dissimilarity measure as a function f(x,y) such that f(x,y) > f(w,z) if and only if x is less similar to y than w is to z.
  Also known as **pair-wise** measure

Distance functions

Different distance functions for

- Different types of data
  - Numeric data
  - Nominal data
- Different applications

Numeric data

- $dist(x_i, x_j)$,where
  - $dist(x_j, x_j) = ((x_{i1} - x_{j1})^h + (x_{i2} - x_{j2})^h + ... + (x_{ir} - x_{jr})^h)^{\frac{1}{h}}$
  - x_i and x_j are data points (vectors)
- Euclidean distance
  - If h = 2, it is the Euclidean distance: $dist(x_j, x_j) = ((x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + ... + (x_{ir} - x_{jr})^2)^{\frac{1}{2}}$
  - Weighted Euclidean distance: $dist(x_j, x_j) = (w_1(x_{i1} - x_{j1})^2 + w_2(x_{i2} - x_{j2})^2 + ... + w_r(x_{ir} - x_{jr})^2)^{\frac{1}{2}}$
  - Squared Eulidean distance: to place progressively greater weight on data points that are further apart: $dist(x_j, x_j) = (x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + ... + (x_{ir} - x_{jr})^2$
- Manhattan distance
  - If h = 1, it is the Manhattan distance: $dist(x_j, x_j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + ... + |x_{ir} - x_{jr}|$
- Minkowski distance (h is positive integer)
  - $dist(x_j, x_j) = ((x_{i1} - x_{j1})^h + (x_{i2} - x_{j2})^h + ... + (x_{ir} - x_{jr})^h)^{\frac{1}{h}}$
  - 1st dimension, 2nd dimension, etc.

- Chebychev distance: define 2 data points as "different" if they are different on any one of the attributes
  - $dist(x_j, x_j) = max(|x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + ... + |x_{ir} - x_{jr}|)$

Binary and nominal Attributes

- Binary attribute: has 2 values or states but no ordering relationship (e.g., gender)
- We use a confusion matrix to introduce the distance function/measures
- Let the i-th and j-th data points be X_i and x_j (vectors)

Confusion matrix

		Data point j		
		1	0	
Data point i	1	a	b	a + b
	0	c	d	c + d
		a + c	b + d	a + b + c + d

- a: the number of attributes with the value of 1 for both data points
- b: The number of attributes with which $x_{if} = 1$ and $x_{jf} = 0$ where x_{if} is the value of f attribute of the data point x_i
- c: the number of attributes of which $x_{if} = 0$ and $x_{jf} = 1$
- d: the number of attributes with the value of 0 for both data points.

Symmetric binary attributes

- A binary attribute is symmetric if both of its states (0 and 1) have equal importance, and carry the same weights, e.g., Gender.
- distance function: **Simple Matching Coefficient**, proportion of mismatches of their values

$$dist(x_i, x_j) = \frac{b + c}{a + b + c + d}$$

Asymmetric binary attributes

- if one of the states is more important or more valuable than the other
  - By convention, state 1 represents the more important state.

- Jaccard coefficient is popular measure

$$dist(x_i, x_j) = \frac{b + c}{a + b + c}$$

- We can have some variations, adding weights

Nominal attributes: with more than 2 states or values.
* the common used distance measure is also based on the simple matching method.
* Given two data points x_i, x_j let the number of attributes be r, and the number of values that match in x_i and x_j be q. $dist(x_i, x_j) = \frac{r-q}{r}$

Data standardization

- In the Euclidean space, standardization of attributes is recommended so that all attributes can have equal impact on the computation of distances.

- standardize attributes: to force the attributes to have a common value range.

- Real data have different types:
  - Interval-scaled
  - symmetric binary
  - asymmetric binary
  - ratio-scaled
  - ordinal
  - nominal

- Convert to a single type:
  - To deal with mixed attributes:
    * Decide the dominant attribute type
    * Convert the other types to this type

30.1.2 K-mean Algorithm

K-means is a partitional clustering algorithm.

Let the set of data points (or instances) D to be $(x_1, ..., x_n)$, where $x_i = (c_{i1}, ..., x_{ir})$ is a vector in a real-valued space $X \subseteq R^r$ and r is the number of attributes (dimensions) in the data

The k-means algorithm partitions the given data into k clusters

- Each cluster has a cluster center (centroid)

- k is specified by user.

Steps

1. Randomly choose k data points (seeds) to be the initial centroids (cluster centers)
2. Assign each data point to the closest centroid
3. Re-compute the centroids using the current cluster memberships
4. If a convergence criterion is not met, go back to 2.

Convergence Criterion

1. No(or minimum) re-assignments of data points to different clusters,
2. No (or minimum) change of centroids, or
3. minimum decreases in the sum of squared error (SSE)

$$SSE = \sum_{j=1}^{k} \sum_{x \in C_j} dist(x.m_j)^2$$

- C is the j-th cluster,m is the centroid of cluster C_j (the mean vector of all the data points in C). and dist(x,m) is the distance between data point x and centroid m_j.

Pros

- Simple: easy to understand and implement
- Efficient: time complexity: O(tkn), where n is the number of data points, k is the number of clusters, and t is the number of iterations.
- Since both ka nd t are small, k-means is considered a linear algorithm
- Note: it terminates at a local optimum if SSE is used. The global optimum is hard to find due to complexity.

Cons

- The algorithm is only applicable if the mean is defined.
  - For categorical data, k-mode
- The user needs to specify k
- The algorithm is sensitive to outliers.
  - outliers could be errors in the data recording or some special data points with very different values.
- is not suitable for discovering clusters that are not hyper-ellipsoids (or hyper-spheres)

Solution

- To deal with outliers:
  - remove some data points int the clustering process that are much further away from the centroids than other data points.
  - Or perform random sampling. Since in sampling we only choose a small subset of the data points, the chance of selecting an outlier is very small.
- To deal with sensitive algorithm
  - Use different seeds

Chapter 31

Semi-superivsed learning

- Goal: exploit both labeled and unlabeled data to build better models than using each one alone
- Self-training
  - Iteratively label some unlabeled examples with a model learned from the previously labeled examples
  - Active learning

Chapter 32

Reinforcement learning

- Learning from interactions
- Goal: learn to choose sequence of actions (= policy) that maximizes $r_0 + \gamma r_1 + \gamma^2 r_2 + ...$

Chapter 33

Minimum Distance

Chapter 34

Quantile Regression

For academic review on quantile regression, check (Yu et al., 2003)

Linear Regression is based on the conditional mean function $E(y|x)$

In Quantile regression, we can view each points in the conditional distribution of y. Quantile regression estimates the conditional median or any other quantile of Y.

In the case that we're interested in the 50th percentile, quantile regression is median regression, also known as least-absolute-deviations (LAD) regression, minimizes $\sum_i |e_i|$

Properties of estimators β

- Asymptotically normally distributed

Advantages

- More robust to outliers compared to OLS
- In the case the dependent variable has a bimodal or multimodal (multiple humps with multiple modes) distribution, quantile regression can be extremely useful.
- Avoids parametric distribution assumption of the error process. In another word, no assumptions regarding the distribution of the error term.
- Better characterization of the data (not just its conditional mean)
- is invariant to monotonic transformations (such as log) while OLS is not. In another word, $E(g(y)) = g(E(y))$

Disadvantages

- The dependent variable needs to be continuous with no zeroes or too many repeated values.

$$y_i = x_i'\beta_q + e_i$$

Let $e(x) = y - \hat{y}(x)$, then $L(e(x)) = L(y - \hat{y}(x))$ is the loss function of the error term.

If $L(e) = |e|$ (called absolute-error loss function) then $\hat{\beta}$ can be estimated by minimizing $\sum_i |y_i - x_i'\beta|$

More specifically, the objective function is

$$Q(\beta_q) = \sum_{i:y_i \geq x_i'\beta}^{N} q|y_i - x_i'\beta_q| + \sum_{i:y_i < x_i'\beta}^{N} (1-q)|y_i - x_i'\beta_q$$

where $0 < q < 1$

The sum penalizes $q|e_i|$ for under-prediction and $(1-q)|e_i|$ for over-prediction

We use simplex method to minimize this function (cannot use analytical solution since it's non-differentiable). Standard errors can be estimated by bootstrap.

The absolute-error loss function is symmetric.

Interpretation For the jth regressor (x_j), the marginal effect is the coefficient for the qth quantile

$$\frac{\partial Q_q(y|x)}{\partial x_j} = \beta_{qj}$$

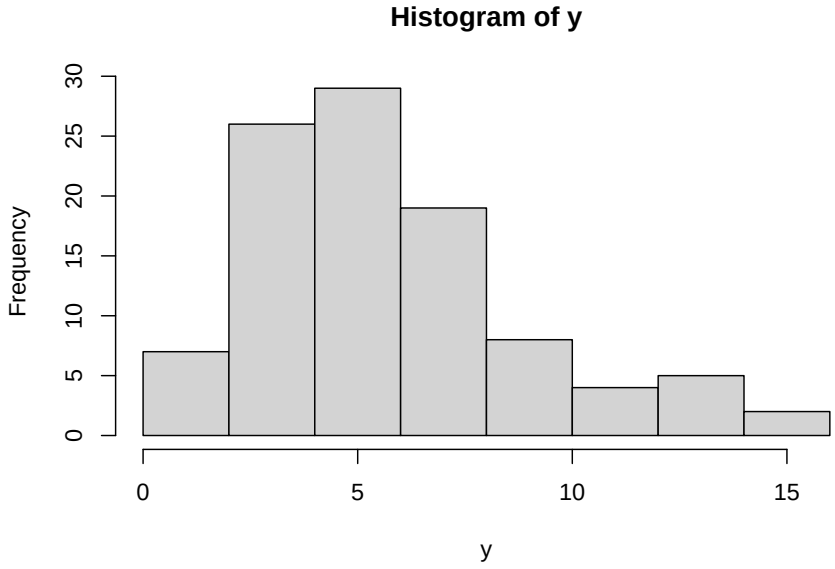
At the quantile q of the dependent variable y, β_q represents a one unit change in the independent variable x_j on the dependent variable y.

In other words, at the qth percentile, a one unit change in x results in β_q unit change in y.

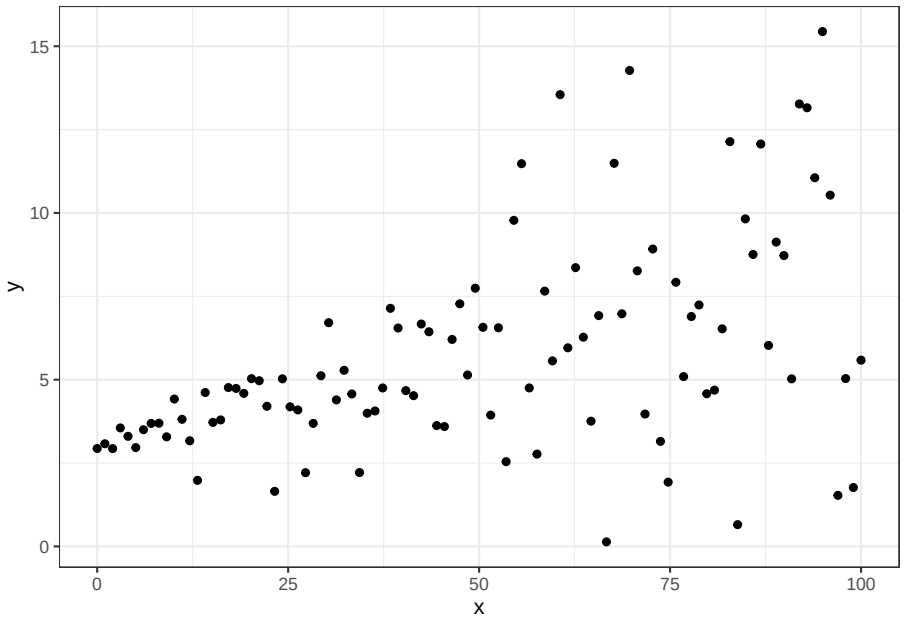
34.1 Application

```
# generate data with non-constant variance

x <- seq(0,100,length.out = 100)        # independent variable
sig <- 0.1 + 0.05*x                     # non-constant variance
b_0 <- 3                                # true intercept
b_1 <- 0.05                             # true slope
set.seed(1)                             # reproducibility
e <- rnorm(100,mean = 0, sd = sig)      # normal random error with non-constant varian
y <- b_0 + b_1*x + e                    # dependent variable
```

```
dat <- data.frame(x,y)
hist(y)
```

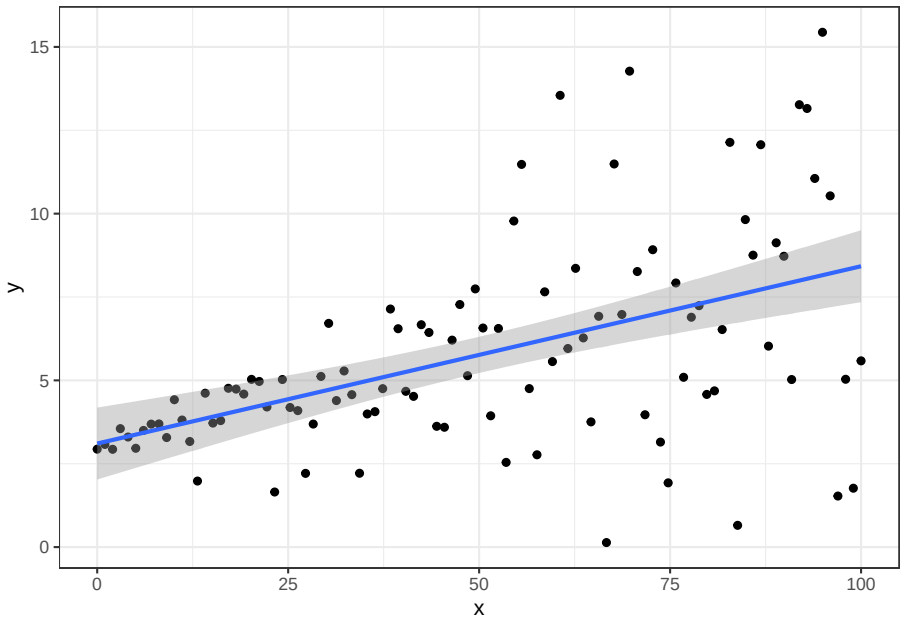



```
library(ggplot2)
ggplot(dat, aes(x,y)) + geom_point()
```




```
ggplot(dat, aes(x,y)) + geom_point() + geom_smooth(method="lm")
```

```
## `geom_smooth()` using formula 'y ~ x'
```



We follow (Koenker, 1996) to estimate quantile regression

```
library(quantreg)
```

```
## Warning: package 'quantreg' was built under R version 4.0.5
```

```
## Loading required package: SparseM
```

```
##
## Attaching package: 'SparseM'
```

```
## The following object is masked from 'package:base':
##
##     backsolve
```

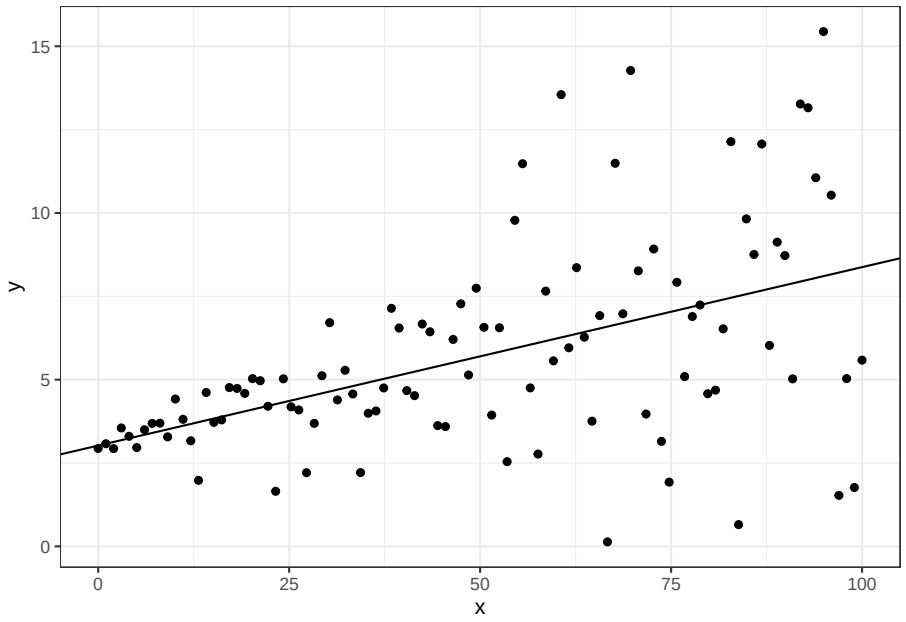
```
qr <- rq(y ~ x, data=dat, tau = 0.5) # tau: quantile of interest. Here we have it at 50th percent
summary(qr)
```

```
## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique
```

```
##
## Call: rq(formula = y ~ x, tau = 0.5, data = dat)
##
## tau: [1] 0.5
##
## Coefficients:
##             coefficients lower bd upper bd
## (Intercept) 3.02410      2.80975  3.29408
## x           0.05351      0.03838  0.06690
```

adding the regression line

```
ggplot(dat, aes(x,y)) + geom_point() +
  geom_abline(intercept=coef(qr)[1], slope=coef(qr)[2])
```



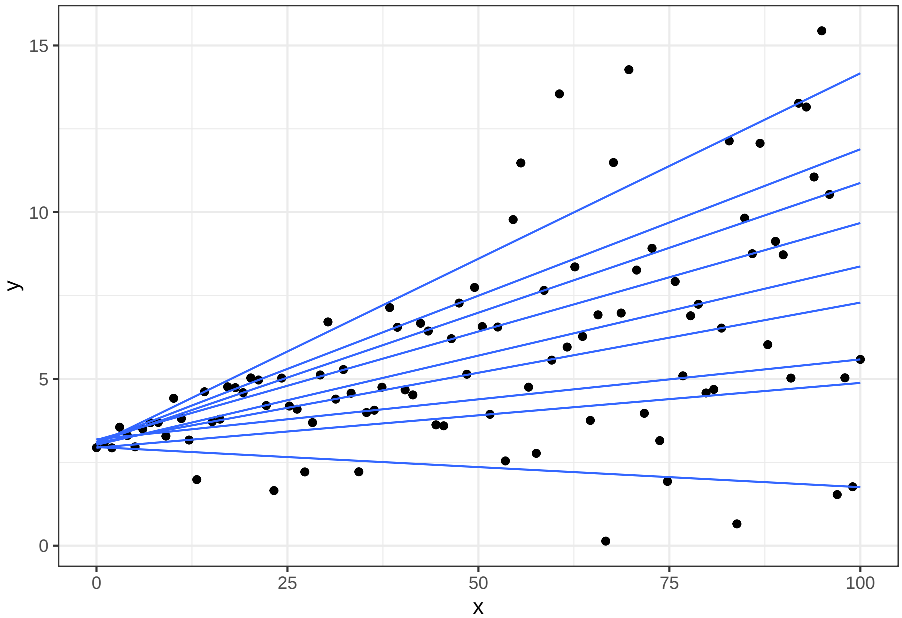
To have R estimate multiple quantile at once

```
qs <- 1:9/10
qr1 <- rq(y ~ x, data=dat, tau = qs)
#check for its coefficients
coef(qr1)
```

```
##                tau= 0.1   tau= 0.2   tau= 0.3   tau= 0.4   tau= 0.5   tau= 0.6
## (Intercept)  2.95735740 2.93735462 3.19112214 3.08146314 3.02409828 3.16840820
## x           -0.01203696 0.01942669 0.02394535 0.04208019 0.05350556 0.06507385
##                tau= 0.7   tau= 0.8 tau= 0.9
## (Intercept) 3.09507770 3.10539343 3.041681
## x           0.07783556 0.08782548 0.111254
```

```
# plot
ggplot(dat, aes(x,y)) + geom_point() + geom_quantile(quantiles = qs)
```

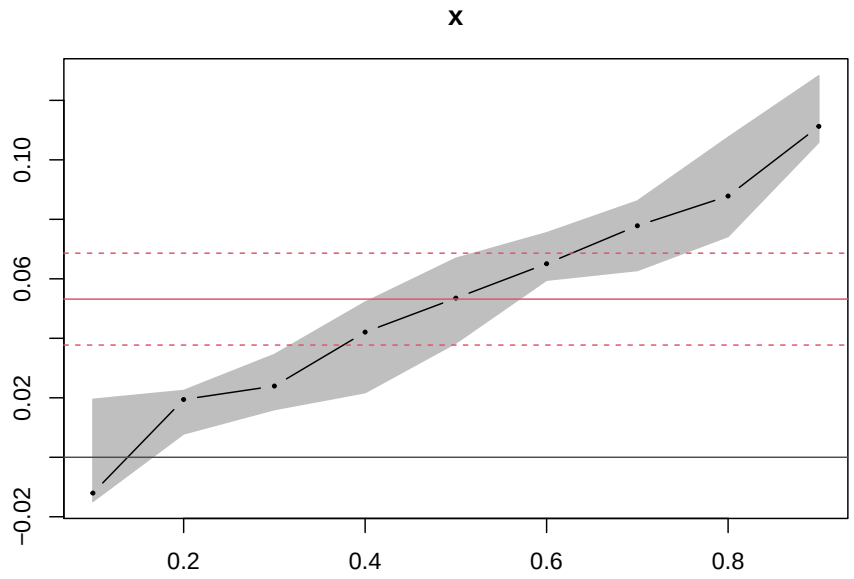
```
## Smoothing formula not specified. Using: y ~ x
```



To examine if the quantile regression is appropriate, we can see its plot compared to least squares regression

```
plot(summary(qr1), parm="x")
```

```
## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique
```



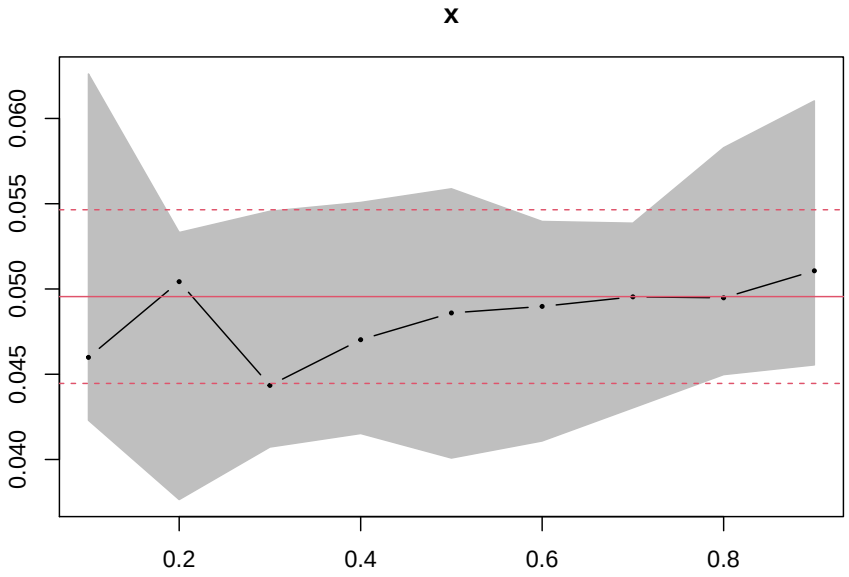
where red line is the least squares estimates, and its confidence interval. x-axis is the quantile y-axis is the value of the quantile regression coefficients at different quantile

If the error term is normally distributed, the quantile regression line will fall inside the coefficient interval of least squares regression.

```
# generate data with constant variance

x <- seq(0, 100, length.out = 100)     # independent variable
b_0 <- 3                               # true intercept
b_1 <- 0.05                            # true slope
set.seed(1)                            # reproducibility
e <- rnorm(100, mean = 0, sd = 1)      # normal random error with constant variance
y <- b_0 + b_1 * x + e                 # dependent variable
dat2 <- data.frame(x, y)
qr2 = rq(y ~ x, data = dat2, tau = qs)
plot(summary(qr2), parm = "x")
```

```
## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique
```

```
## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique

## Warning in rq.fit.br(x, y, tau = tau, ci = TRUE, ...): Solution may be nonunique
```



Bibliography

Freund, Y. and Schapire, R. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139.

Furnival, G. and Wilson, R. (1974). Regressions by leaps and bounds. *Technometrics*, 16(4):499–511.

George, E. and McCulloch, R. (1993). Variable selection via gibbs sampling. *Journal of the American Statistical Association*, 88(423):881–889.

Glorot, X., Bordes, A., and Bengio, Y. (2011). Deep sparse rectifier neural networks. *International Conference on Artificial Intelligence and Statistics*.

Hastie, T., Friedman, J., and Tibshirani, R. (2001). *The Elements of Statistical Learning*. Springer New York.

Koenker, R. (1996). *Quantile Regression*.

Kuhn, M. (2014). Futility analysis in the cross-validation of machine learning models.

Landsman, V. and Stremersch, S. (2020). The commercial consequences of collective layoffs: Close the plant, lose the brand? *Journal of Marketing*, 84(3):122–141.

LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444.

Mahajan, V., Sharma, S., and Buzzell, R. D. (1993). Assessing the impact of competitive entry on market expansion and incumbent sales. *Journal of Marketing*, 57(3):39.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., and Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*. ACM.

Rasmussen, C. and Williams, C. (2005). *Gaussian Processes for Machine Learning*. The MIT Press.

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288.

Wikle, C., Zammit-Mangion, A., and Cressie, N. (2019). *Introduction to Spatio-Temporal Statistics*, pages 1–16. Chapman and Hall/CRC.

Yu, K., Lu, Z., and Stander, J. (2003). Quantile regression: Applications and current research areas. *Journal of the Royal Statistical Society*, 52:331–350.